Large Scale and Multi-Structured Databases
Group Project - FootBall Social Club

Simone Maccarrone - Carlo Pio Pace - Reza Almassi

2024/2025

Contents

List of Figures

Chapter 1

Introduction

1.1 Background and Motivation

Football is the world's most popular sport, captivating billions with its dynamic interplay of skill, strategy, history, and passionate communities. In the digital era, the hunger for data-driven insights, advanced analytics, and socially connected experiences has transformed how fans, analysts, and professionals engage with the game. Yet, the landscape of football data remains fragmented-spread across disparate sources, siloed in varying formats, and often disconnected from the social and analytical features that modern users expect.

The **Football Social Club** project was conceived to address this gap, building an integrated, scalable, and extensible platform that unites rich football datasets, advanced analytics, and community-driven features. By merging traditional data modeling with state-of-the-art NoSQL and graph database technologies, this project sets out to redefine how football data is collected, explored, and experienced.

1.2 Project Vision and Objectives

The primary vision of the Football Social Club platform is to create a unified web-based environment where users, ranging from casual fans to data scientists, can explore, analyze, and interact with a comprehensive spectrum of football data. This platform is designed not only to deliver deep player and team analytics, but also to foster social engagement through articles, discussions, and dynamic relationship networks.

Chapter 2

Data

The data leveraged in this project originates from several sources, primarily an earlier relational database. As such, it initially adhered to a normalized relational schema. Adapt this data to our necessities, underwent a transformation process, where it was redisegned into a denormalized format suitable for the application necessities. This transition involved building and executing multiple operations to reshape the data into document-oriented structures, more aligned with MongoDB's flexible schema design, in order to have a final structure more suitable to deal with the most common queries, that must be satisfied by the system.

2.1 Original Dataset

A detailed list will follow:

- **Football Data**: https://www.kaggle.com/datasets/stefanoleone992/ea-sports-fc-24-complete-player-dataset (110 MB in total)
    1. **180.021** male_players documents;
    2. **6947** male_teams documents;
    3. **1369** male_coaches documents;
    4. **5035** female_players documents;
    5. **231** female_teams documents;
    6. **94** female_coaches documents;
- **Articles Data**: https://www.kaggle.com/datasets/hammadjavaid/football-news-articles (87 MB in total)
    1. **800** allfootball documents;
    2. **1909** df_analyst documents;
    3. **11908** final-articles documents;
    4. **7569** goal-news documents;
    5. **40** live_mint documents;
    6. **278** skysports documents;
    7. **1500** tribuna_articles documents;

Chapter 3

Design

3.1 Overview

This chapter details the architectural and design decisions made for the Football Social Club project. The design is guided by the need for scalability, flexibility, and maintainability.

3.2 Actors

The main actors in the Football Social Club system are:

- **Admin**: Manages all the entities, and data integrity. Responsible for CRUD on master data.
- **Registered User**: Can log in, view info about players, teams, coaches, post/read-/like articles.
- **Unregistered User**: Has limited access, such as viewing basic public, but cannot interact or modify data.

3.3 Requirements

3.3.1 Unregistered Users (Guests)

Functional Requirements

- FR-1:Must be able to create an account, specifying username, email,password and nationality.
- FR-2: Must be able to search for basic information about teams, players, coaches, and users.

3.3.2 Registered Users (Authenticated)

Functional Requirements

- FR-3: A user must be able to log in and log out.

- FR-4: A user must be able to update personal information (Password) and to delete their account.
- FR-5: A user must be able to create, edit, and delete their own articles.
- FR-6: A user must be able to follow other users, like or unlike articles, teams, players and coaches.
- FR-7: A user must be able to search for detailed information about articles, teams, players, coaches, and other users and to see its own ones.
- FR-7: Users must be able to create and manages it's own female/male team, by adding or removing players of any fifa version (corresponds to a version in a specific year) available.

3.3.3 Administrators

Functional Requirements

- FR-8 : Admins must be able to log in and log out.
- FR-9: Admins must be able to view all users, teams, coaches, players, articles and their full details in batch.
- FR-10: Admins must be able to delete user accounts.
- FR-11: Admins must be able to moderate articles, by deleting them if necessary.
- FR-12: Admins must be able to get analytics from the data.
- FR-13: Admins must be able to access audit logs for all administrative actions.
- FR-14: Admins must be able to manually sync data between MongoDB and Neo4j.
- FR-15: Admins must be able to execute CRUD operations on teams, players, and coaches.
- FR-16: Admins must be able to monitor system health and performance.

3.3.4 Non-Functional Requirements

- NFR-1: The system must ensure strong intra-db consistency for administrator write operations. User writes operations will follow an eventual consistency model to improve performance and scalability.
- NFR-2: Databases must handle details, regarding players, teams, and coaches at maximum for 10 years.
- NFR-3: All user passwords must be encrypted using bcrypt.
- NFR-4: The application must be fast and return quick responses to its users in every query.

- NFR-5: The application must be user-friendly for all its users and easy to navigate.

3.4 Mock-Ups

This section shows how the web app is expected to look once fully developed. Each user (or actor) has a personalized categorization tailored specifically to their role.

3.4.1 Unregistered User Manual

In the figure 3.1, are shown the initial view of the app, where an unregistered user can search, and is also possible to register.

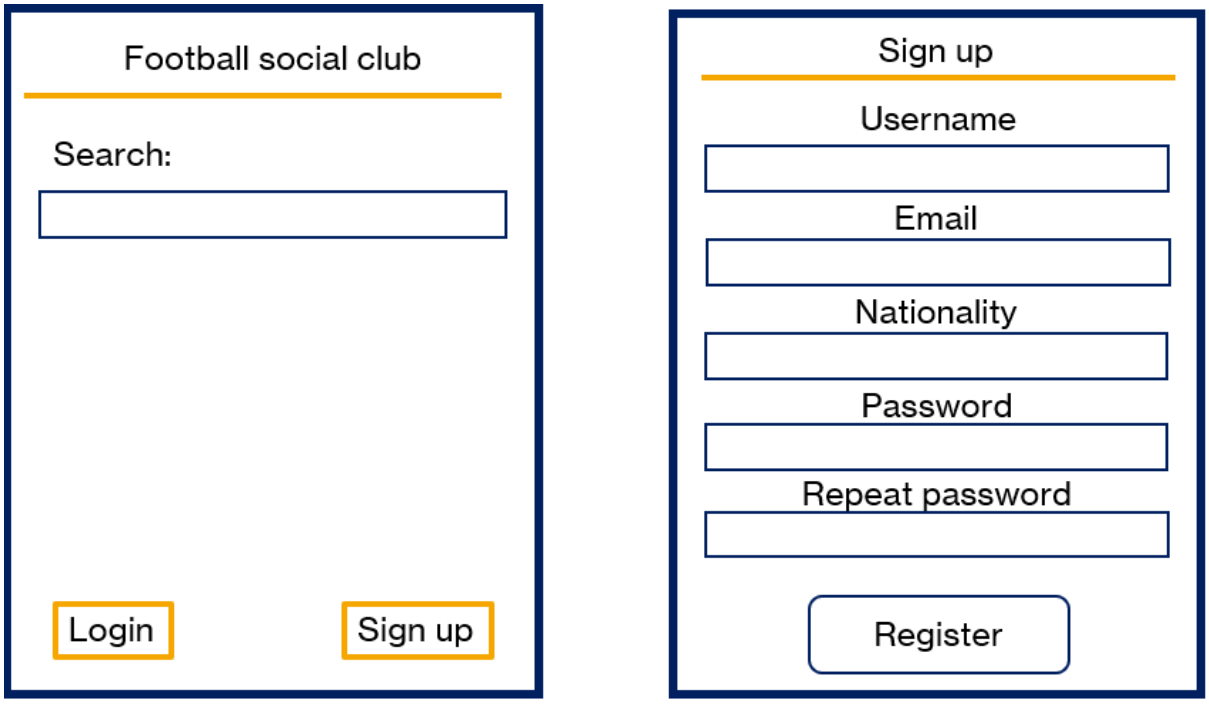

Figure 3.1: Unregistered User Mock Ups (1)

The other two views 3.2 are the result of a search and the visualization of basic info about a user.

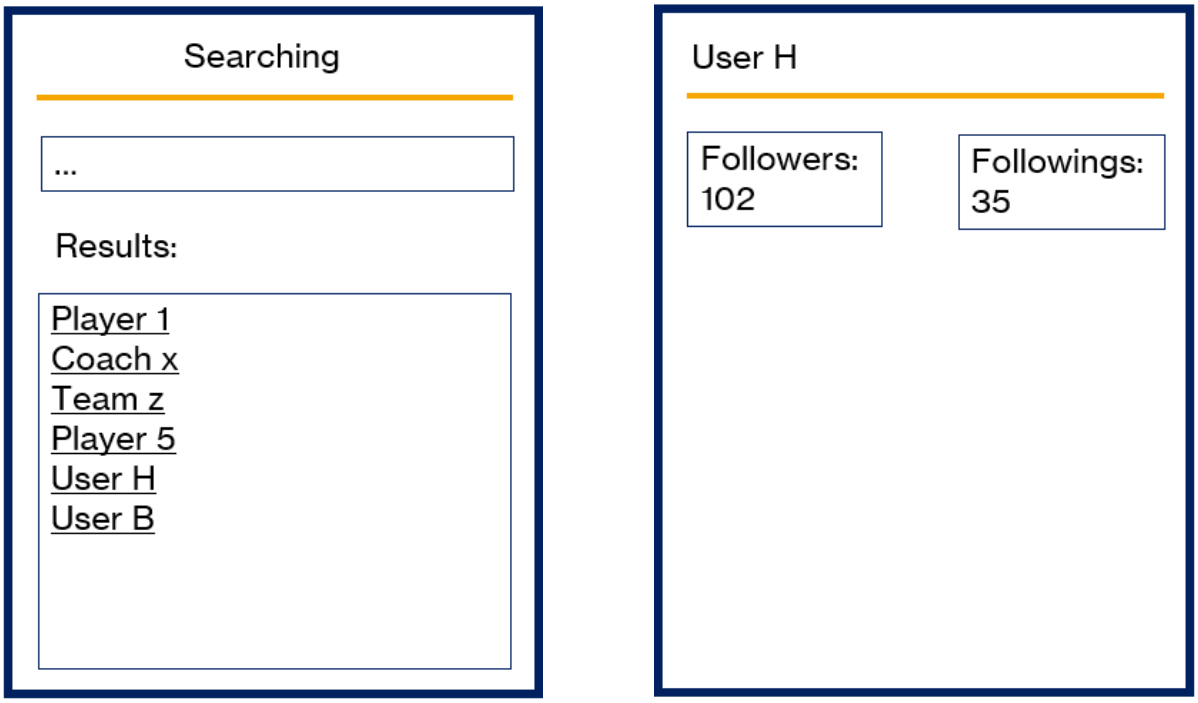

Figure 3.2: Unregistered User Mock Ups (2)

In the block of views 3.3, are shown, what an unregistered user, will see when trying to visualize info about a coach, team or a player.

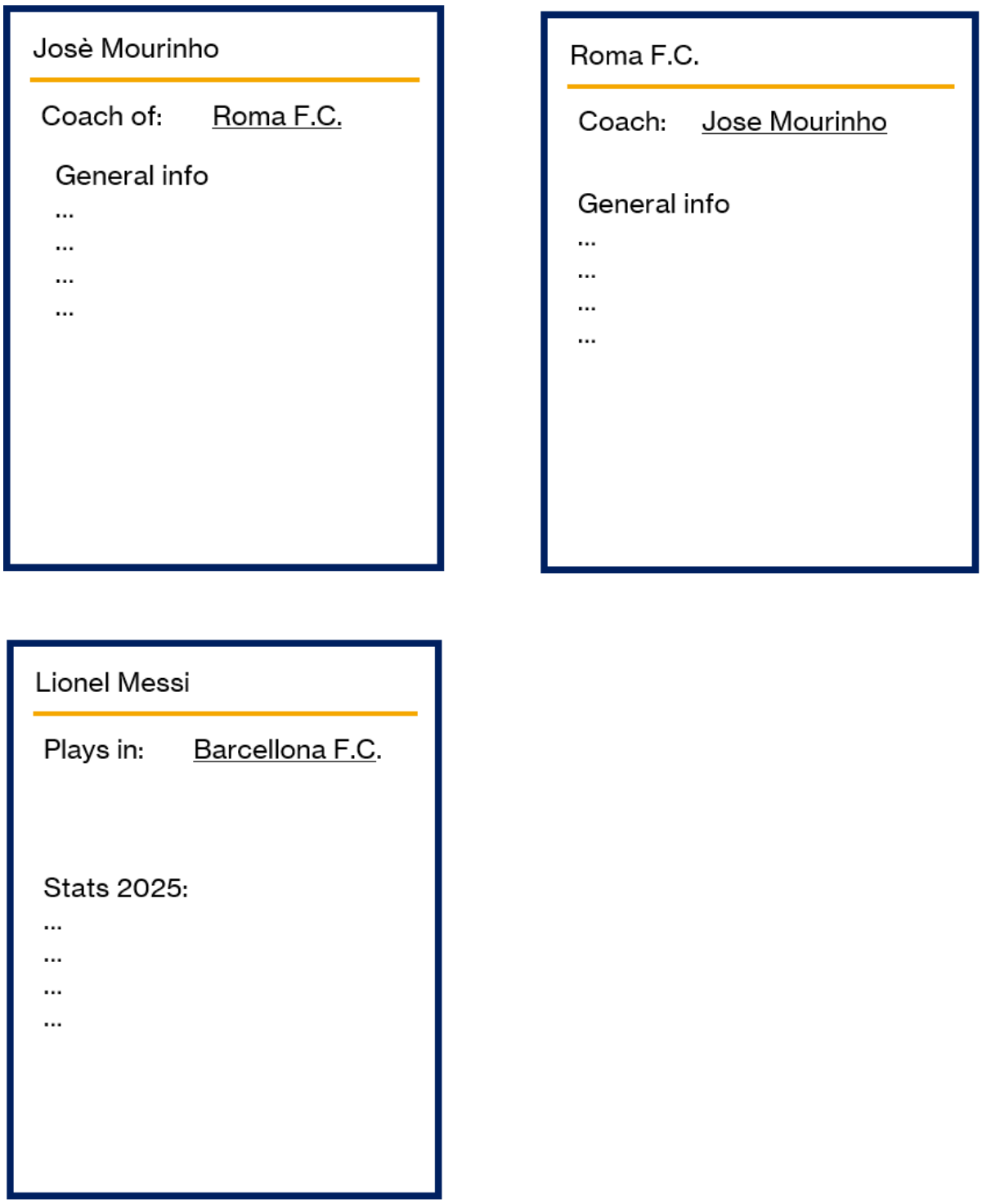

Figure 3.3: Unregistered User Mock Ups (3)

3.4.2 Registered User Manual

In the block of views 3.4 , are shown the login page, also the main page after the login, in which a user can get the list of its followers and followings, can manage its personal team, write an article, get the list of its articles, access the settings view. From the settings view is possible to change the password and also logout. The last view is related , to the personal team when a team has not been created yet.

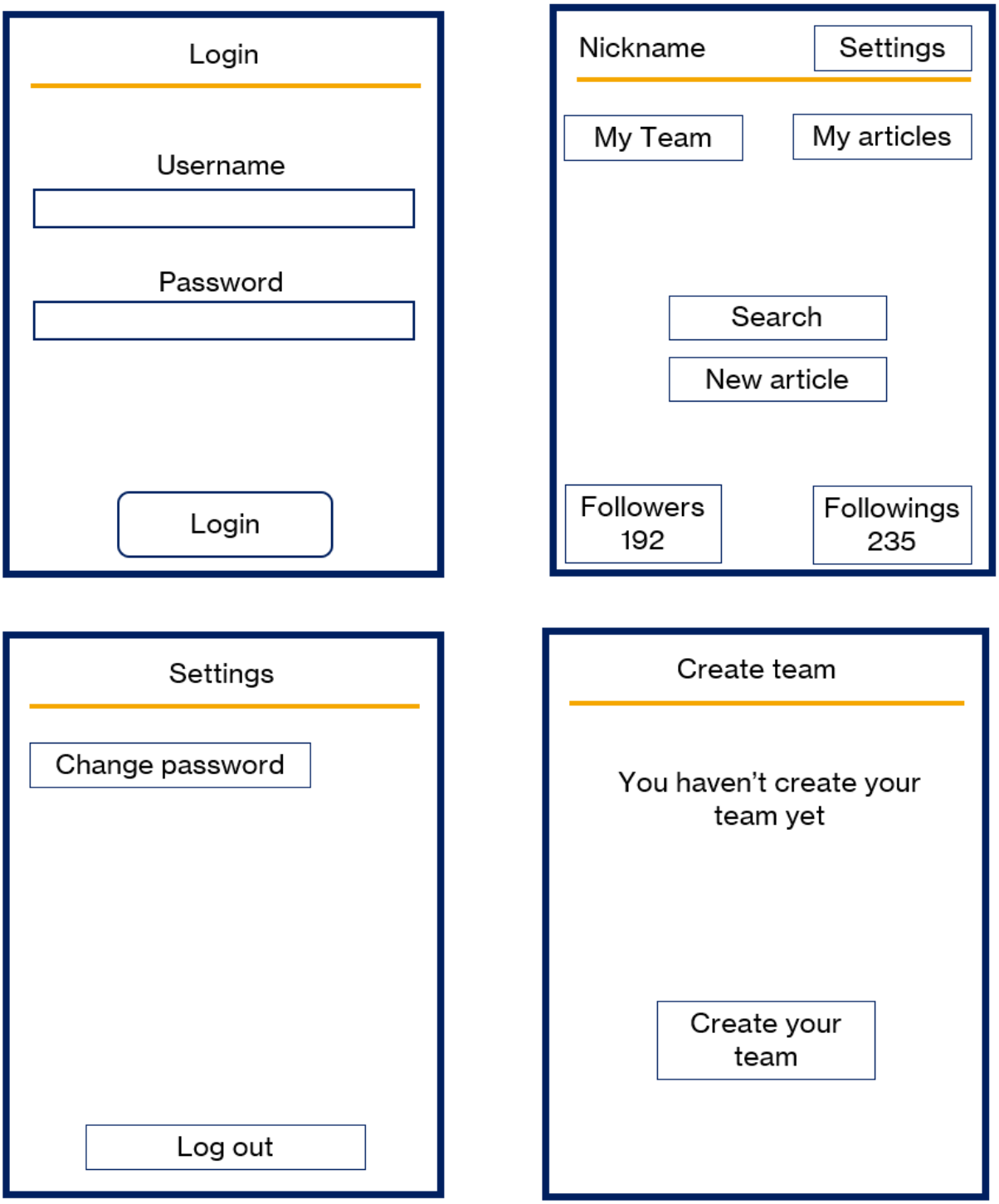

Figure 3.4: Registered User Mock Ups (1)

In the views related to 3.5, are shown some view for managing of the personal team, to add and remove players, and a view for getting the list of written articles, when no articles are written yet.

Figure 3.5: Registered User Mock Ups (2)

In the mock ups 3.6, is possible to see the result of retrieving the list of the written articles, the visualization of an article, the update of an article, and the result of reading the list of the personal followers.

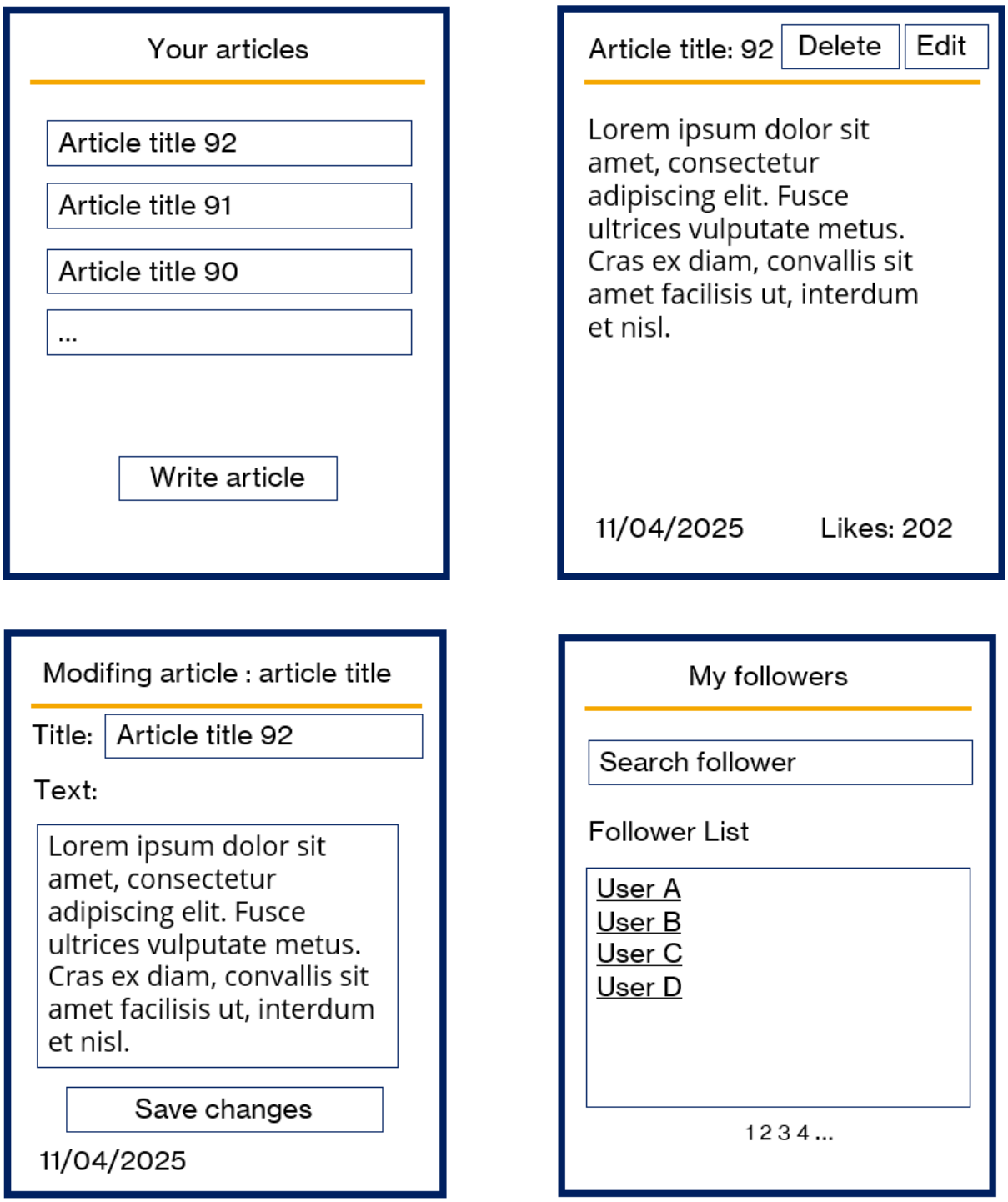

Figure 3.6: Registered User Mock Ups (3)

3.7 shows the mock-up of the result of visualizing the list of personal followings, (which is the same result of viewing the followings of another user), the mock-up of the search section, and the possible filters. Is also present the mock-up related the visualization of another user and showing of details about a coach.

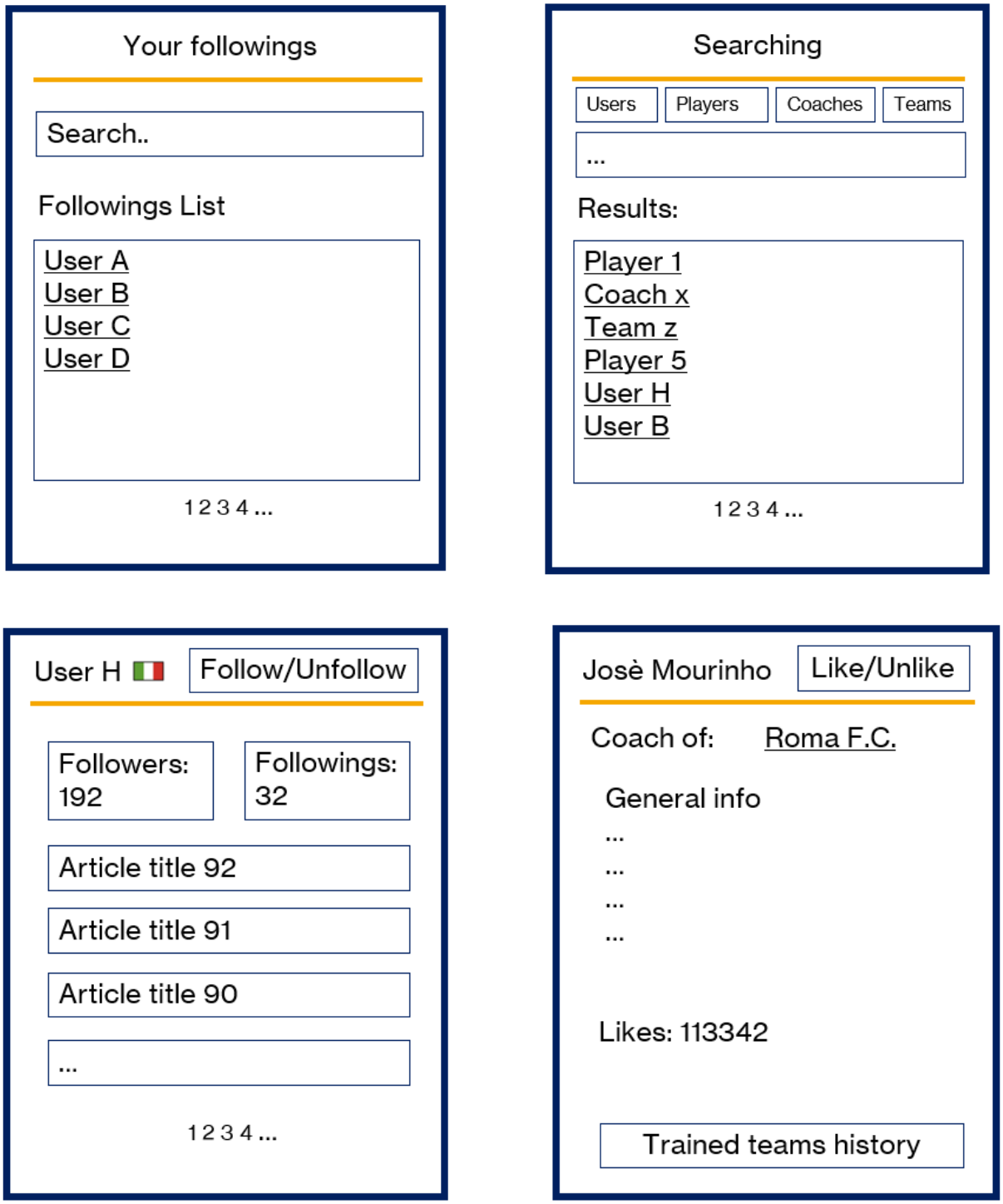

Figure 3.7: Registered User Mock Ups (4)

In 3.8 the first mock up is what an user see when retrieving details about the teams trained by a coach, the second show how the details of a team are shown, following is shown how a formation of a team is visualized. The last one is related to the details of a player.

Figure 3.8: Registered User Mock Ups (5)

3.4.3 Admin Manual

The mock-ups shown in 3.9 in order, the main page seen by an admin, which allows him to access to different sections. The next mock-up allows the admin to see the same information about a player, and check it, but also to check the data about the related node, and also access the section for updating the player or delete them, those sections are the one shown in the following mock ups.

Figure 3.9: Admin Mock Ups (1)

The 3.10 shows, what an admin can see when visualizing a coach, the update and delete sections. The last is related to teams , and the remaining are shown in 3.11, together with the mock-ups related to users.

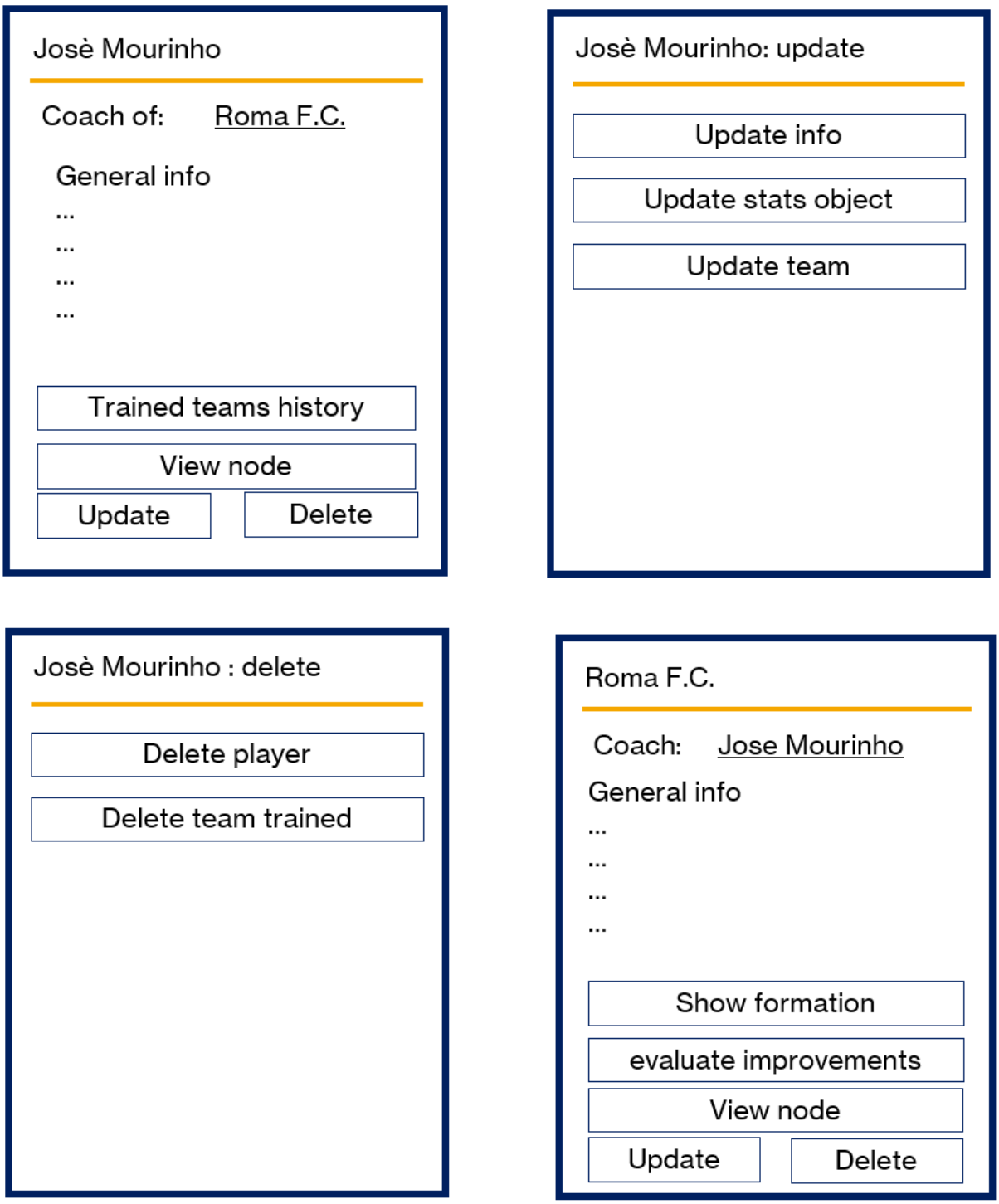

Figure 3.10: Admin Mock Ups (2)

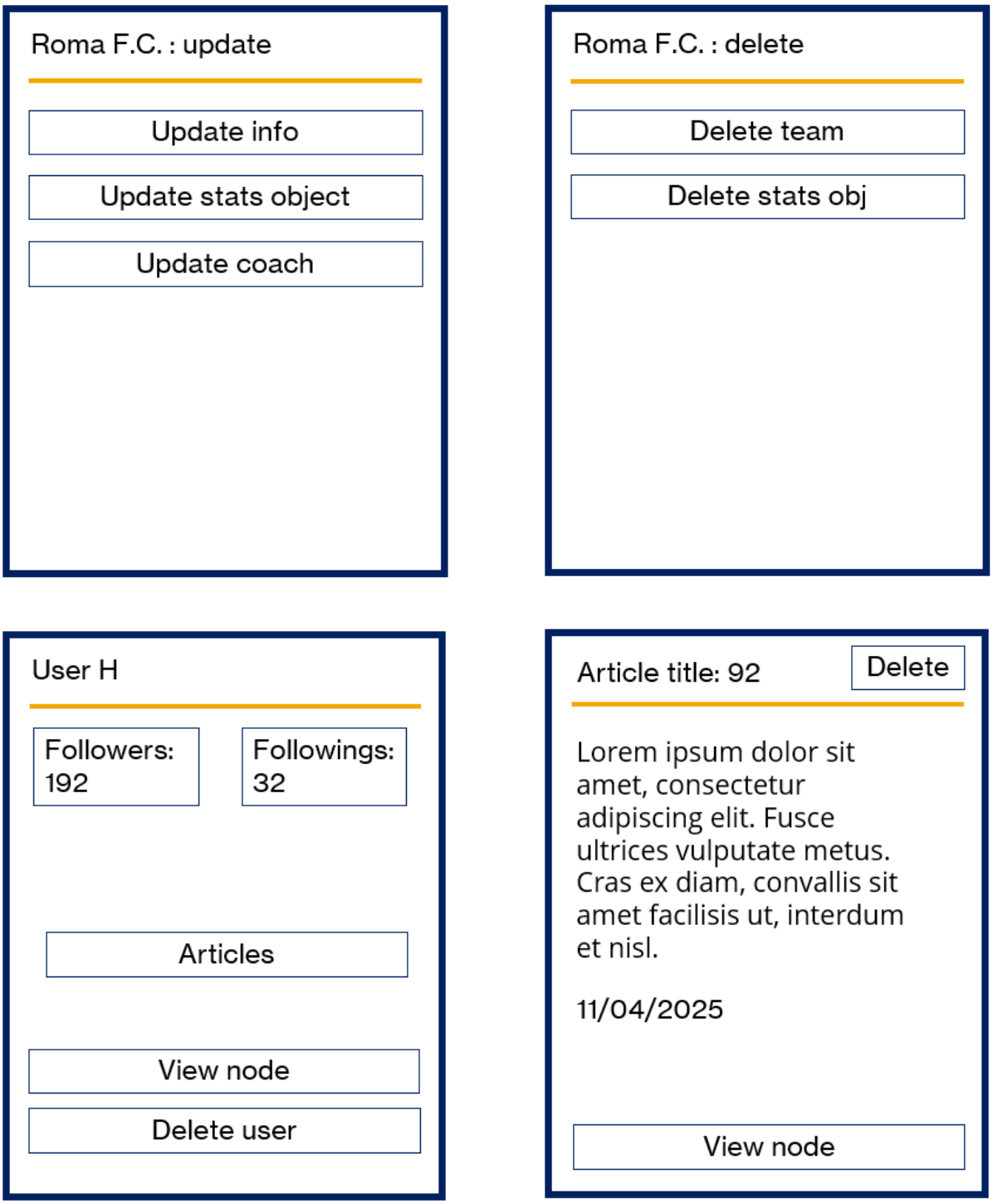

Figure 3.11: Admin Mock Ups (3)

The mock-ups in the 3.12, show the sections in which an admin can add players, coaches and teams, the analytics section also present in 3.13 and 3.14, which also show the section in which are grouped all the mapping operations.

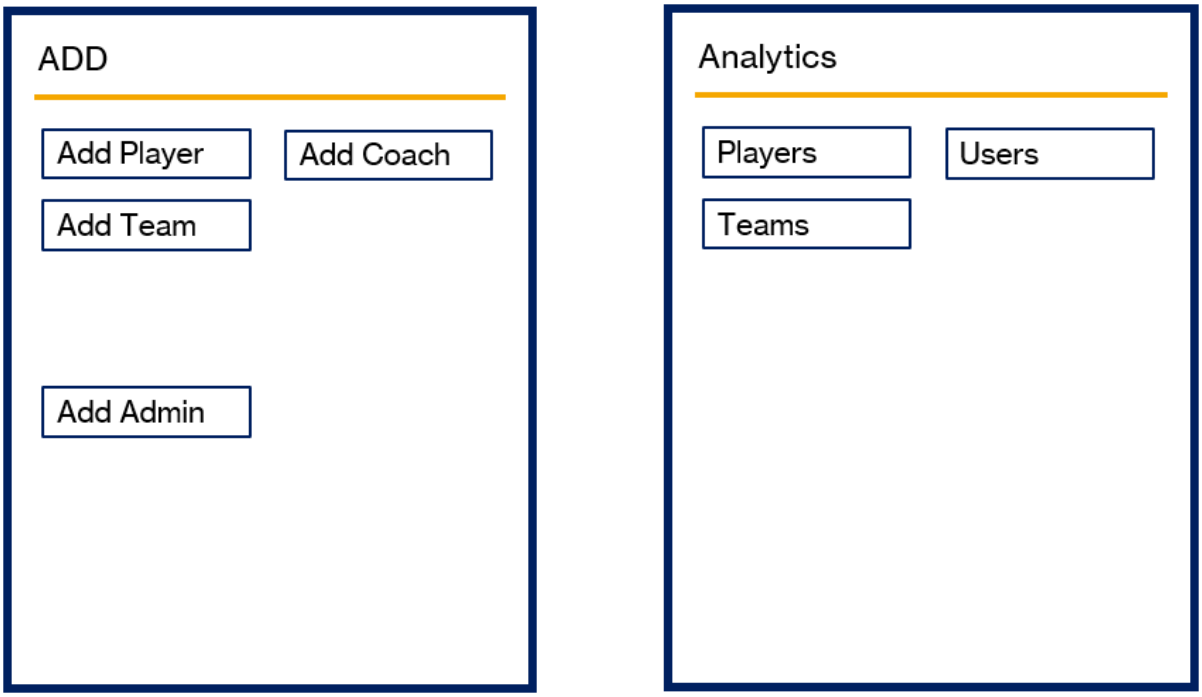

Figure 3.12: Admin Mock Ups (4)

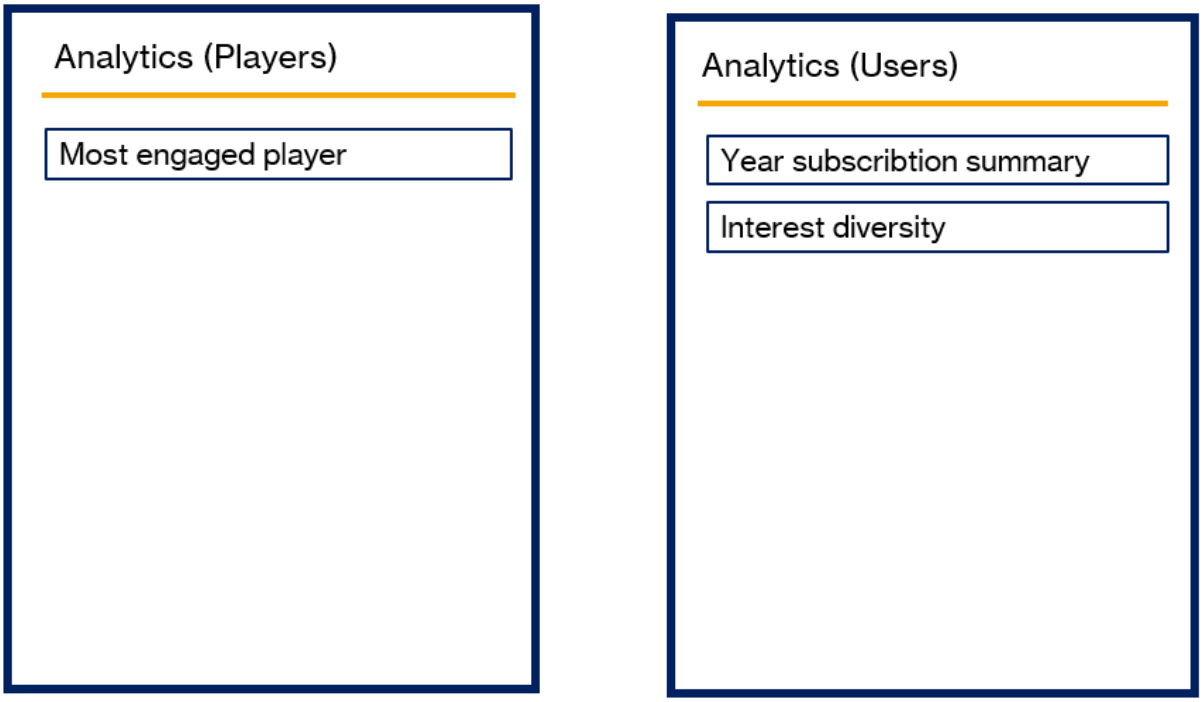

Figure 3.13: Admin Mock Ups (5)

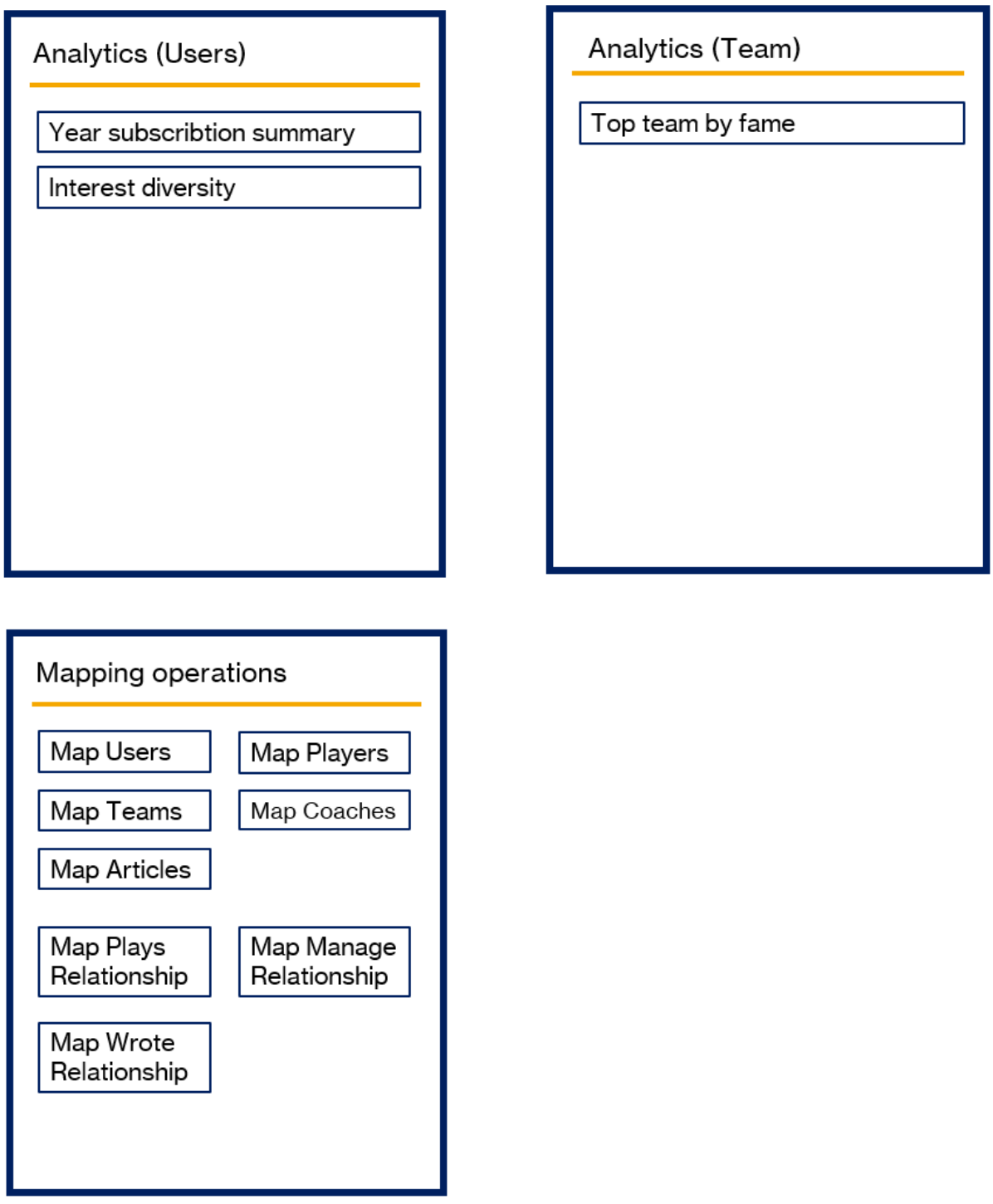

Figure 3.14: Admin Mock Ups (6)

3.5 Dataset Modeling

Starting from the football-related datasets, one of the first steps in the data preparation phase was to unify separate male and female collections into single, consolidated ones. This decision applied across key collections, namely Players, Teams, and Coaches and aimed to simplify queries, avoid normalized structure, exploit duplicate information, and support more flexible analytics. To preserve the original distinction, a new attribute called gender was introduced, allowing for contextual filtering without needing separate schemas. Following this structural unification, a number of attributes were identified as redundant or irrelevant for our application's use cases. These fields, which added unnecessary weight to the documents or were unused in queries or visualizations, were systematically removed to improve performance and ensure cleaner, more maintainable data.

- **Players**:

  player_url, fifa_update, week_foot, skill_moves, international_reputation, real_face, player_tags, and 27 detailed attributes related to specific in-game positions and skills.

- **Coaches**:

  coach_url, nation_flag_url

- **Teams**:

  team_url, fifa_update, rival_team, international_prestige, domestic_prestige, def_style, def_team_width, def_team_depth, off_build_up_play, off_chance_creation

After doing that we de-normalized the collections. For illustration we will present only the Teams pipeline used to reach that result.

```
    db.male_teams.aggregate([
  {
    $lookup: {
      from: "male_coaches",
      localField: "coach_id",
      foreignField: "coach_id",
      as: "coach_info"
    }
  },
  { $unwind: { path: "$coach_info", preserveNullAndEmptyArrays: true }
     },

  {
    $group: {
      _id: "$team_id",
      original_id: { $first: "$_id" },
      team_id: { $first: "$team_id" },
      team_name: { $first: "$team_name" },
      gender: { $first: "male" },
      league_id: { $first: "$league_id" },
      league_name: { $first: "$league_name" },
      league_level: { $first: "$league_level" },
```

```
      nationality_id: { $first: "$nationality_id" },
      nationality_name: { $first: "$nationality_name" },
      fifaStats: {
        $push: {
          fifa_version: "$fifa_version",
          home_stadium: "$home_stadium",
          overall: "$overall",
          attack: "$attack",
          midfield: "$midfield",
          defence: "$defence",
          club_worth_eur: "$club_worth_eur",
          coach: {
            coach_id: "$coach_id",
            coach_mongo_id:{$toString: "$coach_info._id"},
            coach_name: "$coach_info.long_name"
          }
        }
      }
    }
  },
  {
    $set: {
      _id: "$original_id"
    }
  },
  {
    $unset: ["original_id"]
  },
  {
    $merge: {
      into: "Teams",
      whenMatched: "merge",
      whenNotMatched: "insert"
    }
  }
])
```

Listing 3.1: Pipeline to de-normalize male and female teams

As can be seen, information from the older male_teams (also used for female_teams) and male_coaches collections is restructured and merged into a unified **Teams** collection. The operation enriches each team document by embedding relevant coach information, organizing multiple FIFA-versioned stats into a single array, and explicitly tagging the gender as "male" to support later integration with female data.

```
    db.Teams.aggregate([
  // Match Teams that have fifaStats with coach entries
  {
    $match: {
      "fifaStats.coach.coach_id": { $exists: true }
    }
  },

  // Step 1: Extract all coach_ids from fifaStats
  {
    $addFields: {
      coachIds: {
        $setUnion: {
```

```
          $map: {
            input: "$fifaStats",
            as: "fs",
            in: "$$fs.coach.coach_id"
          }
        }
      }
    }
  },

  // Step 2: Lookup canonical coach Mongo IDs from Coaches collection
  {
    $lookup: {
      from: "Coaches",
      let: { ids: "$coachIds" },
      pipeline: [
        {
          $match: {
            $expr: { $in: ["$coach_id", "$$ids"] }
          }
        },
        {
          $project: {
            _id: 1,
            coach_id: 1
          }
        }
      ],
      as: "canonicalCoaches"
    }
  },

  // Step 3: Build map { coach_id -> coach_mongo_id }
  {
    $addFields: {
      coachMap: {
        $arrayToObject: {
          $map: {
            input: "$canonicalCoaches",
            as: "c",
            in: {
              k: { $toString: "$$c.coach_id" },
              v: "$$c._id"
            }
          }
        }
      }
    }
  },

  // Step 4: Update fifaStats[].coach.coach_mongo_id using the map
  {
    $addFields: {
      fifaStats: {
        $map: {
          input: "$fifaStats",
          as: "fs",
          in: {
```

```
            $mergeObjects: [
              "$$fs",
              {
                coach: {
                  $mergeObjects: [
                    "$$fs.coach",
                    {
                      coach_mongo_id: {
                        $let: {
                          vars: {
                            entry: {
                              $first: {
                                $filter: {
                                  input: { $objectToArray: "$coachMap" },
                                  as: "pair",
                                  cond: {
                                    $eq: ["$$pair.k", { $toString: "$$fs.coach.coach_id" }]
                                  }
                                }
                              }
                            }
                          },
                          in: { $toString: "$$entry.v" }
                        }
                      }
                    }
                  ]
                }
              }
            ]
          }
        }
      }
    },

    // Step 5: Cleanup helper fields
    {
      $unset: ["coachIds", "canonicalCoaches", "coachMap"]
    },

    // Step 6: Merge back into Teams collection
    {
      $merge: {
        into: "Teams",
        whenMatched: "merge",
        whenNotMatched: "discard"
      }
    }
]);
```

Listing 3.2: Pipeline to exactly fetch coach informations

After that this aggregation performed a crucial data-cleaning and de-normalization step, specifically enriching and correcting the coach information within each fifaStats entry in the Teams collection. It identifies all referenced coach IDs embedded

in the stats, then maps those to their canonical MongoDB _id values by querying the Coaches collection. This ensures that each coach reference is consistent and properly linked across collections, using unique MongoDB IDs instead of external **coach_ids**. Final steps were the one to clean up key values like coach_id and teams_id, used for the SQL schema, and by adding default FifaStats Object up to 10 to meet the requirements. The final version of the dataset counted:

- **55550** Players documents;
- **1240** Teams documents;
- **1463** Coaches documents;

For what concern indeed the Articles Data, First, the Users collection was created by extracting only the valid authors from the dataset. This step ensured that only correctly identified authors were registered as users.

```
   // Retrieve all distinct authors from the Articles collection
const articles = db.Articles.find();
const authorPatterns = [" / Author","By "," and ","GFFN | "];

// Iterate over each author to create a corresponding user
articles.forEach(article => {
    // Skip entries where the author field and publish time , null or
       empty
    //also some articles don't have an author correctly set
    //instead they have a long phrase
    //if the string is longer than 5 words

    let varAuthor=article.author;


    if (!varAuthor || varAuthor.length==0 || varAuthor.split(" ").
       length>5|| varAuthor=="Author not found") {
        return;
    }


    //removing if present some string patterns
    authorPatterns.forEach(pattern => {
      if (varAuthor.includes(pattern)) {
    varAuthor=varAuthor.replace(pattern,"");
      }
    });

    // Check if a user with the same username already exists
    if (db.Users.findOne({ username: varAuthor })) {
        return;
    }

    // Create the user document with username, random password, and
       role
    db.Users.insertOne({
        username: varAuthor,
        password: generateRandomPassword(),
  signup_date:generateRandomDate(startDate,endDate),
        roles: ["ROLE_USER"]
```

```
    });
});
```

Listing 3.3: Pipeline to create Users

After establishing the Users collection, the Articles dataset was then cleaned by removing articles with authors that were misspelled or not recognizable, improving the overall data quality.

```
    const validUserUsernames = db.Users.find({}, { _id:0,username: 1 })
        .toArray().map(user => user.username);

// Step 2: Delete all articles where the author is not in the list of
    valid users
const result = db.Articles.deleteMany({
  author: { $nin: validUserUsernames }
});

print(`${result.deletedCount} articles deleted.`);
```

Listing 3.4: Pipeline to to delete authors

Following these steps, additional pipelines were run to enhance the data: random hashed passwords, sign-up dates, nationality and e-mails were generated for the users, while random publish times were assigned to the articles, adding realism and completeness to the data. The final collections has:

- **19280** Articles documents;
- **341** Users documents

3.6 Use Case Diagram

The following use case diagram outlines the main interactions within the system.

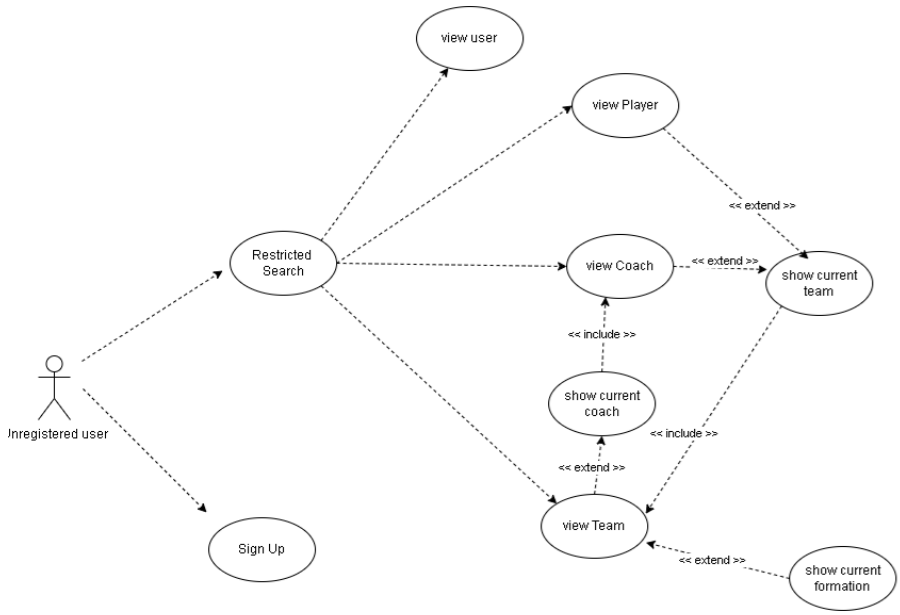

Figure 3.15: Unregistered Users use case diagram

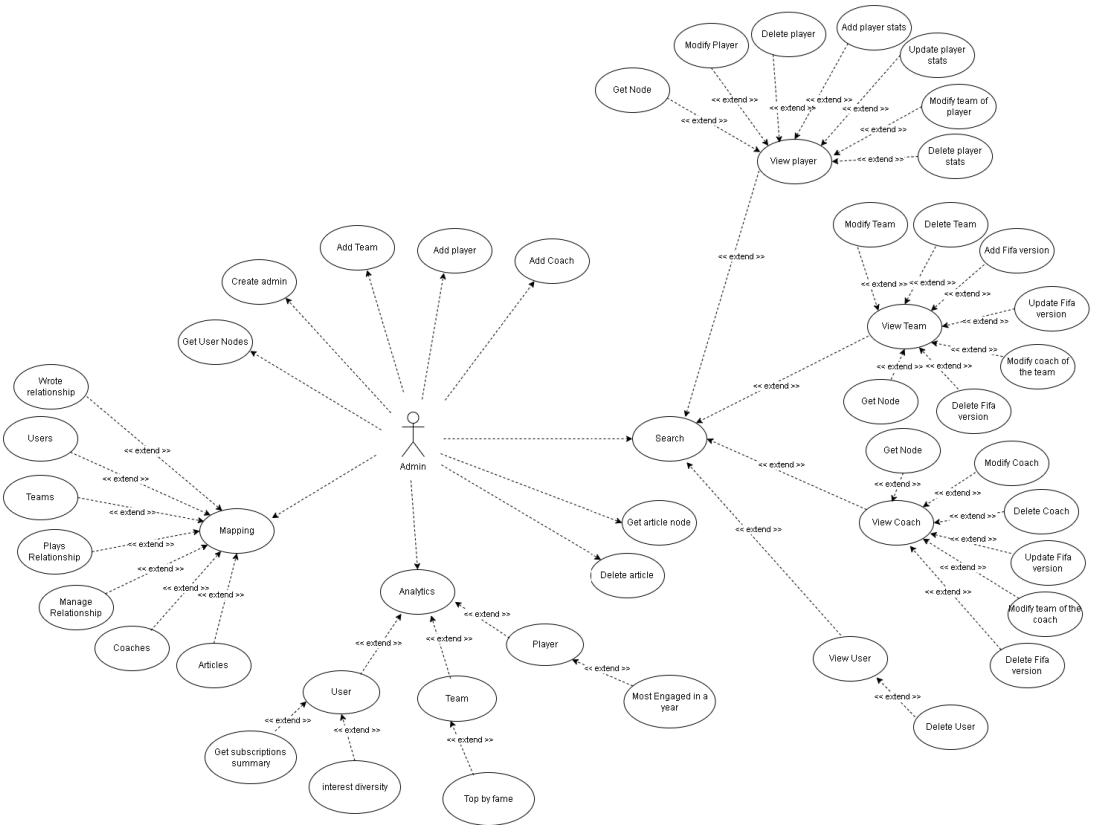

Figure 3.16: Admins use case diagram

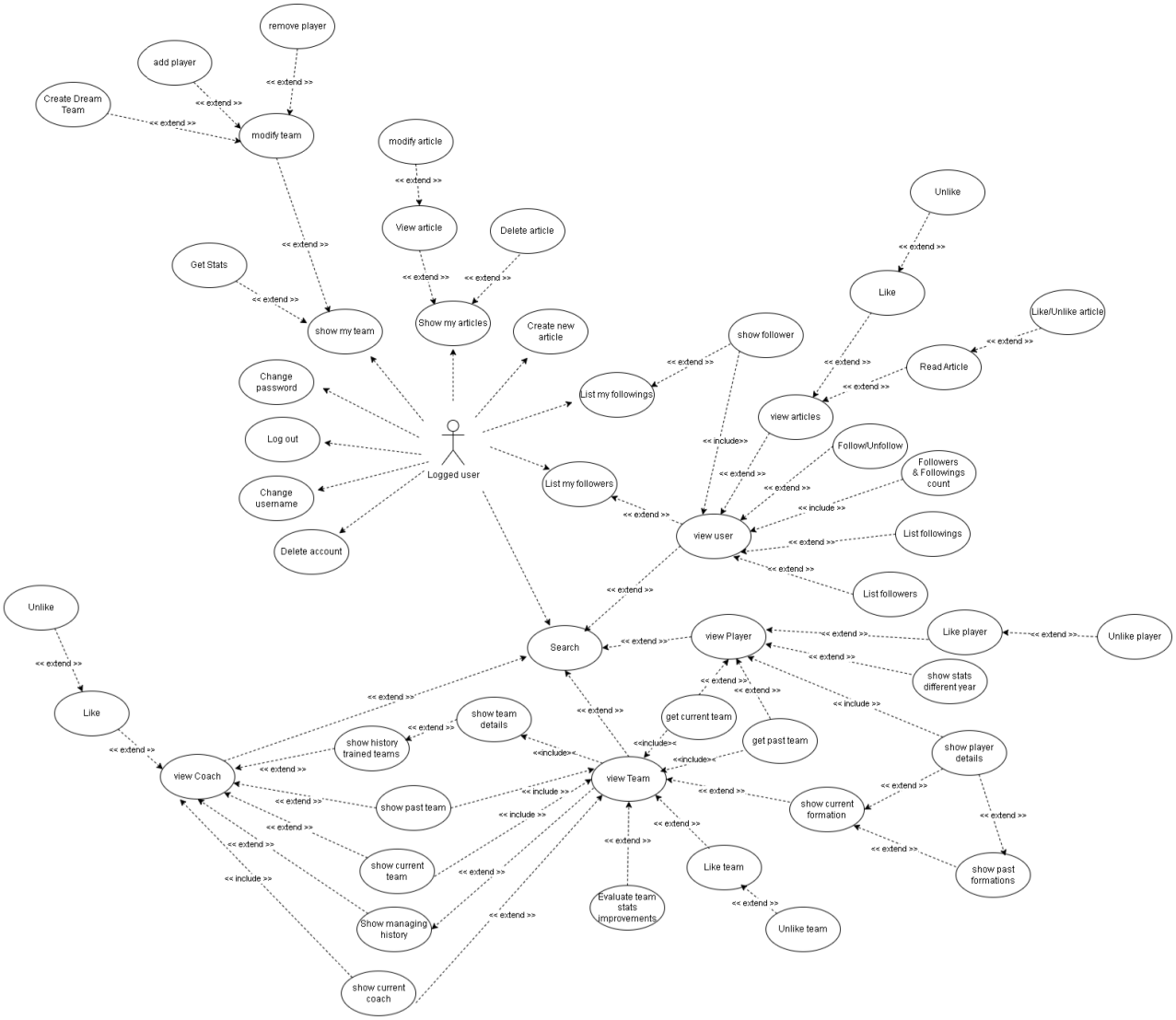

Figure 3.17: Registered Users use case diagram

3.7 Class Diagram

The UML class diagram captures the main domain entities and their relationships.

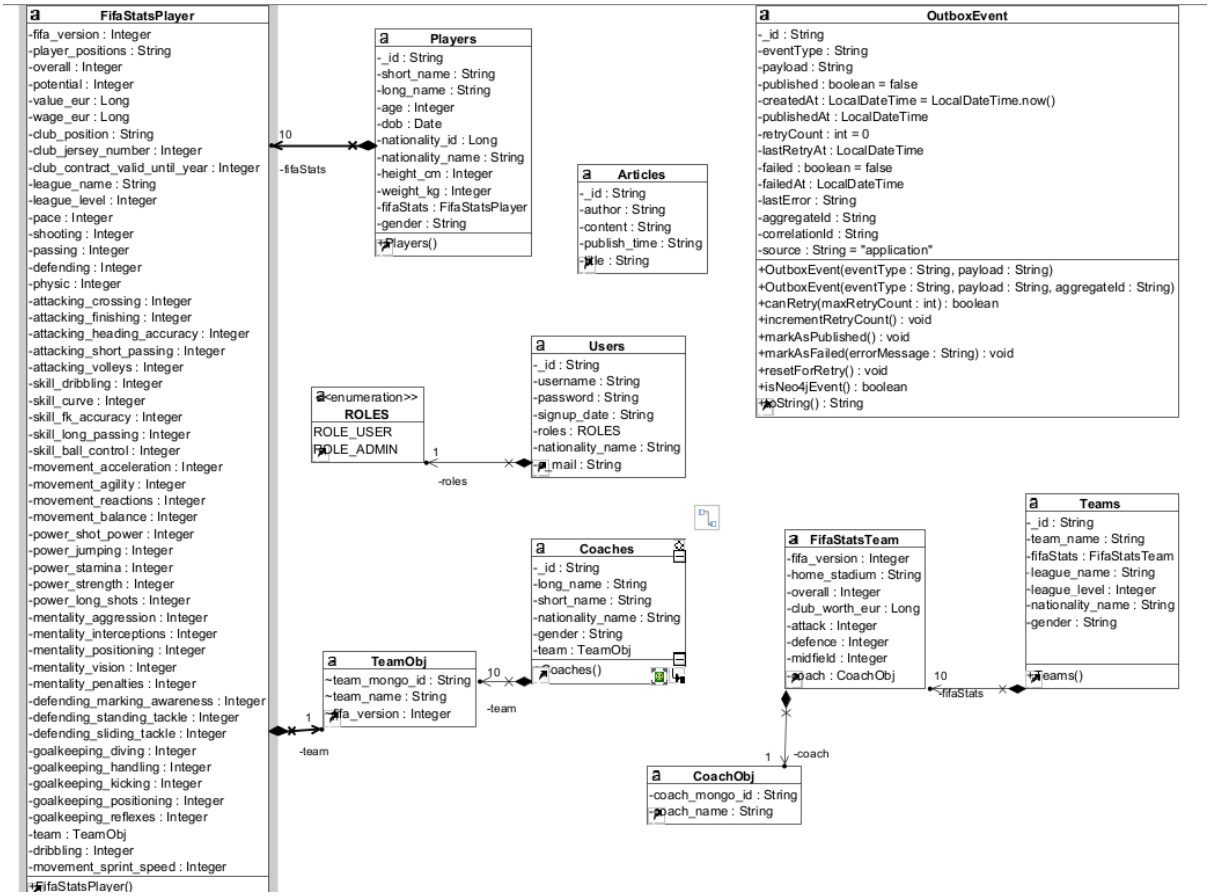

Figure 3.18: MongoDB enitities class diagram

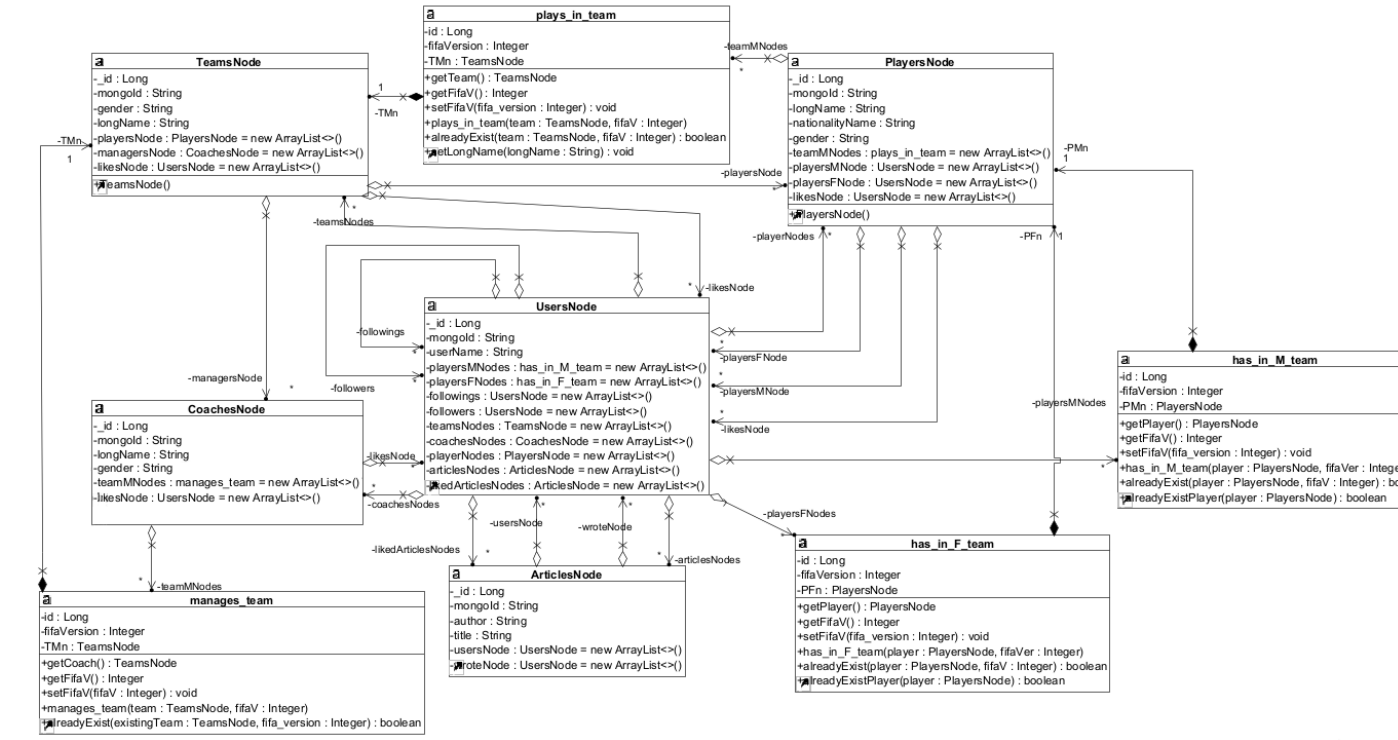

Figure 3.19: Neo4j entities class diagram

3.8 Data Models and Storage

In this section we present the final result of the collections stored in MongoDB presented in Chapter 2 with some examples.

3.8.1 MongoDB Collections

1. **Players Collection**


```
  {
"_id": {
  "$oid": "6836e6adae93aaefe539083e"
},
"age": 21,
"dob": {
  "$date": "2002-05-29T00:00:00.000Z"
},
"fifaStats": [
  {
    "fifa_version": 24,
    "player_positions": "CM",
    "overall": 68,
    "potential": 79,
    "value_eur": 2700000,
    "wage_eur": 500,
```

```
"club_position": "SUB",
"club_jersey_number": 15,
"club_contract_valid_until_year": 2024,
"league_name": "Women's Super League",
"league_level": 1,
"release_clause_eur": 6500000,
"team": {
  "team_mongo_id": "6836e6b7ae93aaefe5391658",
  "team_name": "Liverpool W"
},
"pace": 71,
"shooting": 66,
"passing": 59,
"dribbling": 69,
"defending": 57,
"physic": 61,
"attacking_crossing": 35,
"attacking_finishing": 72,
"attacking_heading_accuracy": 46,
"attacking_short_passing": 69,
"attacking_volleys": 56,
"skill_dribbling": 69,
"skill_curve": 30,
"skill_fk_accuracy": 52,
"skill_long_passing": 64,
"skill_ball_control": 72,
"movement_acceleration": 70,
"movement_sprint_speed": 71,
"movement_agility": 70,
"movement_reactions": 69,
"movement_balance": 65,
"power_shot_power": 59,
"power_jumping": 58,
"power_stamina": 65,
"power_strength": 59,
"power_long_shots": 64,
"mentality_aggression": 60,
"mentality_interceptions": 58,
"mentality_positioning": 70,
"mentality_vision": 69,
"mentality_penalties": 54,
"mentality_composure": 59,
"defending_marking_awareness": 56,
"defending_standing_tackle": 64,
"defending_sliding_tackle": 49,
"goalkeeping_diving": 11,
"goalkeeping_handling": 9,
"goalkeeping_kicking": 9,
```

```
  "goalkeeping_positioning": 13,
  "goalkeeping_reflexes": 6
},
{
  "fifa_version": -1,
  "player_positions": "NA",
  "overall": -1,
  "potential": -1,
  "value_eur": -1,
  "wage_eur": -1,
  "club_position": "NA",
  "club_jersey_number": -1,
  "club_contract_valid_until_year": 2999,
  "league_name": "DefaultLeague",
  "league_level": -1,
  "team": {
    "team_mongo_id": "XXXXXXXXXXXX",
    "team_name": "DefaultTeam"
  },
  "pace": -1,
  "shooting": -1,
  "passing": -1,
  "dribbling": -1,
  "defending": -1,
  "physic": -1,
  "attacking_crossing": -1,
  "attacking_finishing": -1,
  "attacking_heading_accuracy": -1,
  "attacking_short_passing": -1,
  "attacking_volleys": -1,
  "skill_dribbling": -1,
  "skill_curve": -1,
  "skill_fk_accuracy": -1,
  "skill_long_passing": -1,
  "skill_ball_control": -1,
  "movement_acceleration": -1,
  "movement_sprint_speed": -1,
  "movement_agility": -1,
  "movement_reactions": -1,
  "movement_balance": -1,
  "power_shot_power": -1,
  "power_jumping": -1,
  "power_stamina": -1,
  "power_strength": -1,
  "power_long_shots": -1,
  "mentality_aggression": -1,
  "mentality_interceptions": -1,
  "mentality_positioning": -1,
```

```
        "mentality_vision": -1,
        "mentality_penalties": -1,
        "defending_marking_awareness": -1,
        "defending_standing_tackle": -1,
        "defending_sliding_tackle": -1,
        "goalkeeping_diving": -1,
        "goalkeeping_handling": -1,
        "goalkeeping_kicking": -1,
        "goalkeeping_positioning": -1,
        "goalkeeping_reflexes": -1
      },
      // 8 other default objects
    ],
    "gender": "female",
    "height_cm": 172,
    "long_name": "Sofie Lundgaard",
    "nationality_id": 13,
    "nationality_name": "Denmark",
    "preferred_foot": "Right",
    "short_name": "S. Lundgaard",
    "weight_kg": 63
  }
```


2. **Teams Collection**


```
    {
    "_id": {
      "$oid": "6836e721ae93aaefe53bddb3"
    },
    "fifaStats": [
      {
        "fifa_version": 24,
        "home_stadium": "Estadio de la Cerámica",
        "overall": 65,
        "attack": 65,
        "midfield": 66,
        "defence": 65,
        "club_worth_eur": 2500000,
        "coach": {
          "coach_mongo_id": "6836e7471fea6d2694941d99",
          "coach_name": "Miguel Álvarez Jurado"
        }
      },
      {
        "fifa_version": 23,
        "home_stadium": "Town Park",
        "overall": 65,
        "attack": 67,
```

```
      "midfield": 65,
      "defence": 65,
      "club_worth_eur": 2500000,
      "coach": {
        "coach_mongo_id": "6836e7471fea6d2694941d99",
        "coach_name": "Miguel Álvarez Jurado"
      }
    },
    {
      "fifa_version": -1,
      "home_stadium": "DefaultStadiumName",
      "overall": -1,
      "attack": -1,
      "midfield": -1,
      "defence": -1,
      "club_worth_eur": -1,
      "coach": {
        "coach_mongo_id": "XXXXXXXXXXXX",
        "coach_name": "DefaultCoachName"
      }
    },
   //other 8 default objects
  ],
  "gender": "male",
  "league_level": 2,
  "league_name": "La Liga 2",
  "nationality_name": "Spain",
  "team_name": "Villarreal II"
}
```


3. **Coaches Collection**


```
    {
  "_id": {
    "$oid": "6836e7471fea6d2694941bc4"
  },
  "gender": "male",
  "long_name": "Josh Wolff",
  "nationality_name": "United States",
  "short_name": "J. Wolff",
  "team": [
    {
      "team_mongo_id": "6836e721ae93aaefe53bdd21",
      "team_name": "Austin",
      "fifa_version": 24
    },
    {
      "team_mongo_id": "6836e721ae93aaefe53bdd21",
```

```
      "team_name": "Austin",
      "fifa_version": 23
    },
    {
      "team_mongo_id": "6836e721ae93aaefe53bdd21",
      "team_name": "Austin",
      "fifa_version": 22
    },
    {
      "team_mongo_id": "XXXXXXXXXXXX",
      "team_name": "DefaultTeamName",
      "fifa_version": -1
    },
   //other 6 default objects
  ]
}
```


4. **Articles Collection**


```
   {
  "_id": {
    "$oid": "681f1de03d005462fc240459"
  },
  "author": "Ameé Ruszkai",
  "title": "'We know how good we are' - Will Sweden end
  England's Women's Euros hopes?",
  "content": "Magdalena Eriksson, the Sweden defender and Chelsea captain,
  was speaking to the press about her nation's
  Women's Euro semi-final opponent,
  England. Next to her was Kosovare Asllani - the team's No.9 by shirt,
  but....
  "publish_time": "07/04/2023"
}
```


5. **Users Collection**


```
{
  "_id": {
    "$oid": "681f1e1be69dc7d7fad0263d"
  },
  "username": "Liam Wraith",
  "password": "328a72925fef8cda822d1b2b5d02e5d50e33f866
  77e4138519f6f7acd9f4138e",
  "signup_date": "05/12/2019",
  "roles": [
    "ROLE_USER"
  ],
  "nationality_name": "Ireland",
```

```
  "email": "LiaWra@gmail.com"
}
```


6. **OutBox_Event Collection**


```
    {
  "_id": {
    "$oid": "683c4e4952f17f677575e494"
  },
  "eventType": "Neo4jFollowUser",
  "payload": "{\"targetUsername\":\"Simone\",\"username\":\"Bruna\"}",
  "published": false,
  "createdAt": {
    "$date": "2025-06-01T12:57:45.337Z"
  },
  "retryCount": 0,
  "failed": false,
  "aggregateId": "Simone",
  "source": "application",
  "_class": "com.example.demo.models.MongoDB.OutboxEvent"
}
```


7. **Failed_Events Collections**


```
    {
  "_id": {
    "$oid": "6835f9602052852ae0a952ea"
  },
  "eventType": "Neo4jFollowUser_FAILED",
  "payload": "{\"targetUsername\":\"dsaafasda\",\"username\":\"Bruna\"}",
  "published": false,
  "createdAt": {
    "$date": "2025-05-27T17:41:52.657Z"
  },
  "retryCount": 999,
  "failed": true,
  "failedAt": {
    "$date": "2025-05-27T17:41:52.657Z"
  },
  "source": "application",
  "_class": "com.example.demo.models.MongoDB.OutboxEvent"
}
```


These last two collections support eventual consistency between MongoDB and Neo4j using Kafka. The OutBox_Event collection temporarily stores domain events triggered by user actions, which are then asynchronously published to Kafka. If the publishing fails after several retries, the event is moved to the Failed_Events collection for monitoring or manual recovery. This ensures data changes are reliably propagated across systems, even in the face of transient failures.

3.8.2 Neo4j Graph Structure

Neo4j models the social/relational aspect. Every node is a 1-1 copy of the MongoDB collection, but with way few attributes.

Nodes

- **PlayersNode**

```
<elementId>: 4:2460e9f5-0a74-40ae-b3d0-580489169de8:12
<id>: 12
age: 29
gender: male
longName: Javier Antonio Grbec
mongoId: 68249add7b8223979a5c3bcc
nationalityName: Argentina
```

- **TeamsNode**

```
<elementId>: 4:2460e9f5-0a74-40ae-b3d0-580489169de8:28622
<id>: 28622
gender: male
longName: Sheffield United
mongoId: 682493307b8223979a59b8f7
```

- **CoachesNode**

```
<elementId>: 4:2460e9f5-0a74-40ae-b3d0-580489169de8:85413
<id>: 85413
gender: male
longName: Marcos Ignacio Ambriz Espinoza
mongoId: 682604e3089e934f5d6bf340
```

- **UsersNode**

```
<elementId>: 4:2460e9f5-0a74-40ae-b3d0-580489169de8:28282
<id>: 28282
mongoId: 681f1e1ce69dc7d7fad0264c
userName: Milestone.
```

- **ArticlesNode**

```
<elementId>: 4:2460e9f5-0a74-40ae-b3d0-580489169de8:86879
<id>: 86879
author: Nick Khairi
mongoId: 681f1de03d005462fc240455
title: Would you wear these in Sunday League?
Adidas unveils limited edition Predator boots emblazoned
with Swarovski crystal design
```

Relationships

1. UsersNode – **FOLLOW** → UsersNode
2. UsersNode – **WROTE** → ArticlesNode
3. UsersNode – **LIKES** → ArticlesNode/PlayersNode/CoachesNode/TeamsNode
4. UsersNode – **HAS_IN_M_TEAM {fifa_version}** → PlayersNode
5. UsersNode – **HAS_IN_F_TEAM {fifa_version}** → PlayersNode
6. PlayersNode – **PLAYS_IN_TEAM {fifa_version}** → TeamsNode
7. CoachesNode – **MANAGES_TEAM {fifa_version}** → TeamsNode

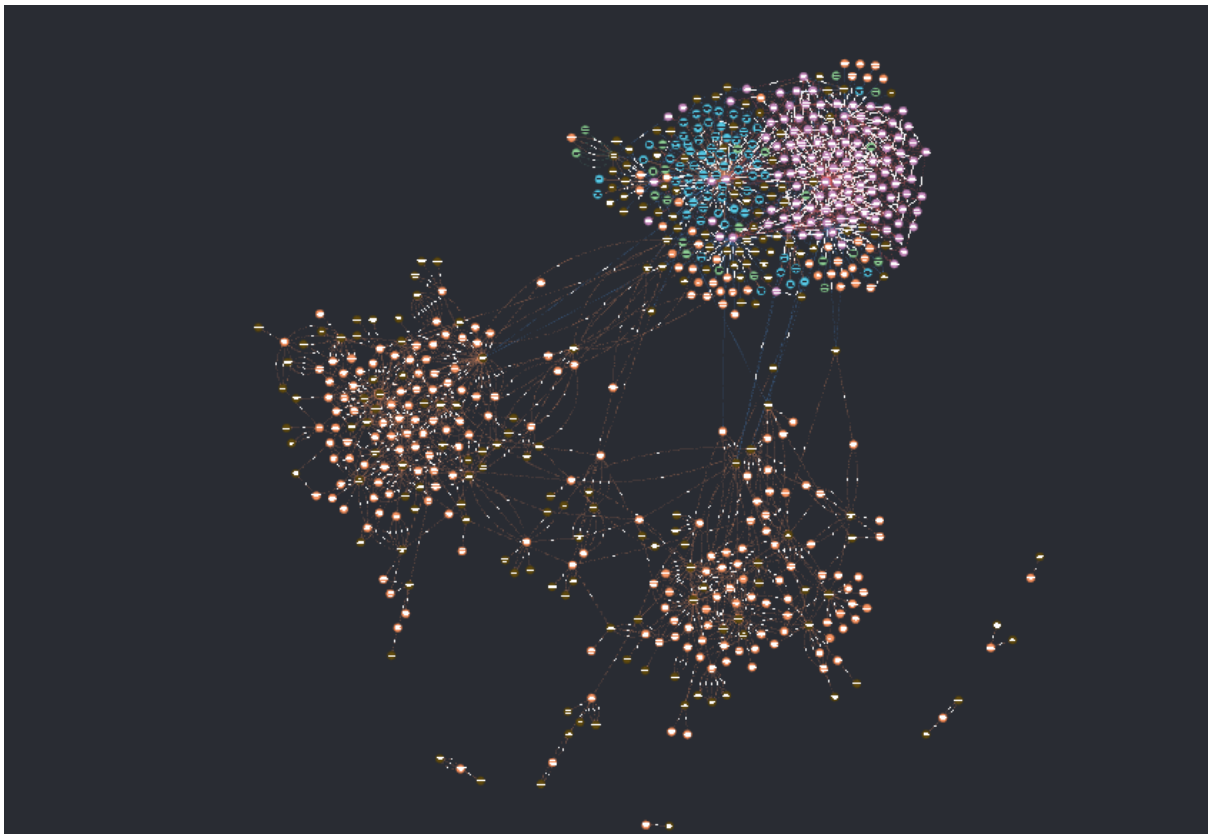

Figure 3.20: Neo4j Structure - In the picture PlayersNode are orange, UsersNodes are pink and ArticlesNodes are blue.

3.8.3 Volume Considerations

In this section, we analyze the expected data growth and storage requirements associated with the system. The goal is to provide a probable scenario in which the system could operate, anticipating how the data may grow in the future.

Football Data Growth

The dataset was originally scraped from [1], a website developed by Khachin Borjigin, which is popular among FIFA game players and football enthusiasts. The developer utilized a service provided by the company SportMonks [2], which offers accurate football-related data. Since SportMonks has several partnerships with official football federations, we decided to base our database growth predictions on the annual expansion of their data. The following estimations were computed based on the average new data per collection added for each fifa version (corresponds to a year). This because of the requirements that limits the number of versions to be stored for each entity:

Table 3.1: Average new entities added and Storage Size growth per year

Collections	Male	Female	Avg. Coll. Size (kB)	Total Size (MB)
Players	241	196	12,49	5,45
Teams	8	9	2,39	0,042
Coaches	9	9	1,07	0,019
Total	258	214	15,95	5,5

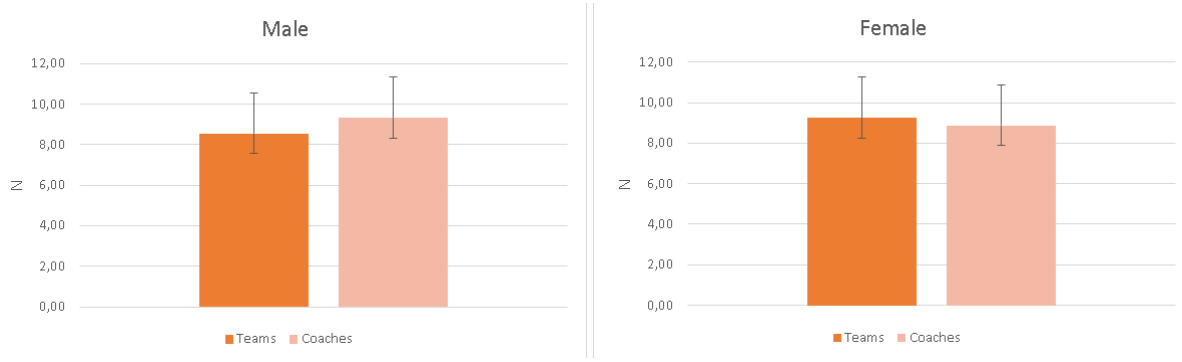

Figure 3.21: Column Chart Male and Female Coaches and Team avg growth by year

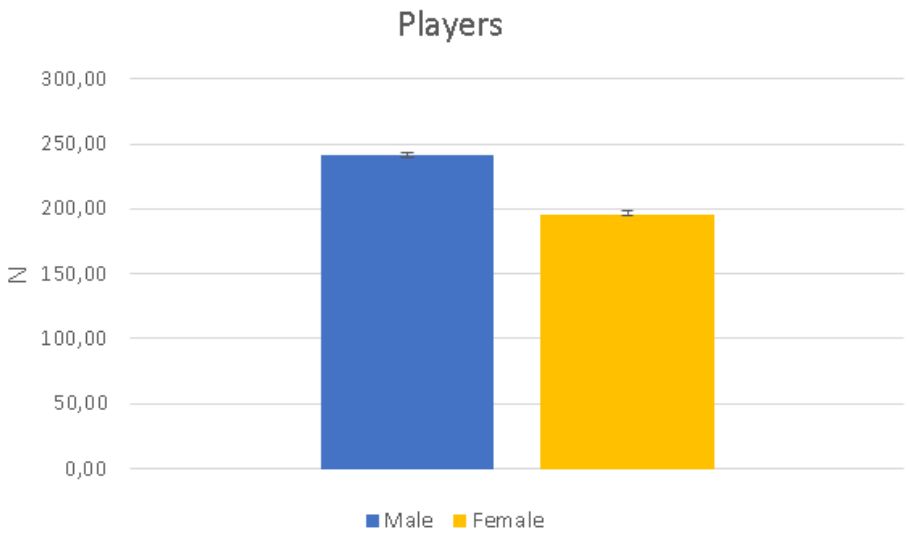

Figure 3.22: Column Chart Players avg growth by year

Table 3.2: Avarage new nodes and relationships added and Storage Size growth per year

Nodes and Relationships	Male	Female	Avg. Coll. Size (kB)	Total Size (MB)
PlayersNode	241	196	0,53	0,23
TeamsNode	8	9	0,4	0,007
CoachesNode	9	9	0,53	0,009
PLAYS_IN_TEAM	2410	1960	0,075	0,18
MANAGES_TEAM	90	90	0,075	0,007
Total	2758	2264	1,61	0,43

Users and Contents growth

Regarding the social-related growth, our projections are grounded in several authoritative sources. According to the 2024 UEFA Annual Report [3], a total of 221 million spectators attended European club football matches (live) during the 2024 season. Complementing this, data from Eurostat [5] reveals that the European Union had a population of approximately 449.3 million in 2024, with 63% of individuals falling within the 15-64 age range. Furthermore, based on statistics from [4], 88% of people aged 16–29 in the EU actively use smartphones and social media platforms.

Building on these insights, we define a conservative (lower-bound) estimation for user growth. Specifically, we hypothesize that 5% of the population aged 18–50 who attend football matches will begin using our application within the next five years. This adoption trend is modeled using an exponential growth function:

$$U(t) = (1 + r)^t$$

Where:

- $U(t)$ is the number of users at time t;

- t represents time in months;
- r is the monthly exponential growth rate.

Assuming a total of 10 million registered users after five years, we estimate a potential monthly exponential growth rate of approximately 24%.

- As for the user-generated articles, based on the findings in VidMob studies [7], we assume that 10% of users are "Active," publishing one article per week, while 40% are considered "Semi-Active," publishing one article per month. The remaining 50% are treated as "Passive" users who only consume content.
- As for the likes and follows interaction, we based our computation on the analytics of SemRush [9] and [10], [11], [12], [13], counting 568 milions monthly active users on X, with an avarage of 707 followers per user, and 2 bilion monthly active users on Istagram, with an avarage of 395 likes per post.
- For the "create team" functionality provided for users we will do an upper bound estimation, considering that all the user will have 11 players in the male team and same for the female one.

Table 3.3: Avarage new entities added and Storage Size growth per year

Collection	New Documents	Avg. Document. Size (kB)	Total Size (MB)
Users	1.729.851	0,25	432
Articles	17.299.197	3,19	55.000
Total	19.029.048	3,44	55.432

Table 3.4: Avarage new nodes and relationships added and Storage Size growth per year

Nodes&Relationships	New	Avg. Size (kB)	Total Size (MB)
UsersNode	1.729.851	0,27	467
ArticlesNode	17.299.197	0,4	6.900
WROTE	17.299.197	0,03	519
HAS_IN_F_TEAM	19.028.361	0,075	1.400
HAS_IN_M_TEAM	19.028.361	0,075	1.400
FOLLOWS	20.759.037	0,03	623
LIKES	34.598.395	0,03	1,04
Total	129.706.399	1,05	16.605

Note:

For the avarage documents size we referred to the automatic computation made from MongoDB Desktop application, while for Neo4j entities avarage size we referred to the offical documentation [8].

3.9 Database Indexing

To ensure high performance and efficient query execution across large datasets, indexes were used strategically in the the system. This section outlines the indexes used in MongoDB and Neo4j to optimize frequent read operations, enforce uniqueness, and support scalability, especially under high concurrency and large-scale data conditions.

3.9.1 MongoDB Indexes

In MongoDB, no indexes outside the default mongoId, have been used, this apparently controversial decision, is the result of design considerations and also tests (see chapter 4.4). The considerations are:

- Nature of the queries, the most frequent queries, are carried out using the mongoId on the documents.
- The most important query that don't use the mongoId, is the global search among all collections, but it's mechanism doesn't exploit Indexes.

The global search mechanism, is "retrieve all the documents, which name field **containes** a particular pattern, in any position". This mechanism was chosen, because it's the most suitable to retrieve the expected information, but in this case the indexes cannot be used, since in MongoDB the only way to use indexes in this type of search is using anchored search, meaning that the mechanism should be "retrieve all documents which attribute name **starts** with a pattern". For example if a user search "Messi", in the MongoDB the document related to Messi, has in the name field "Lionel Andrés Messi Cuccittini", so the results with both mechanism will be:

- Using contained pattern, all the documents which contains "messi" in the attribute name;
- Using anchored pattern, only the documents which attribute name, starts with "messi".

So using the anchored pattern obliges users, to know the exact name of the player (same for coaches, teams, users).

Is important to mention that, MongoDB offers other solutions to use Indexes also for the contained patterns, using MongoDB Atlas Search, which was not used in our case.

3.9.2 Neo4j Indexes

In Neo4j , we choose to use 2 indexes, in order to speed up the read queries, those indexes are:

- For users nodes, an index on the username;
- For players, coaches and teams an index on the MondoDB id.

The reasons why these indexes have been adopted are:

- The difference of volumes between the entities, the collections which is the biggest for now, is the players, but since the app is also a social media, in the future, the amount of users will be much more than the player, so the slowest read will be on this type of nodes.
- Most of the queries executed on users are carried out using usernames.
- All the queries which need to find on the Neo4j database,nodes related to player, coaches, teams and articles use the MongoDB id.

3.10 Intra Database error handling

In a multi-database setup leveraging both MongoDB and Neo4j, consistency management, between these 2 DB, becomes a key architectural concern. The system enforces strong consistency for administrative operations, while user-generated actions are handled under eventual consistency using the outbox pattern and Apache Kafka. This chapter explains how write flows are monitored, categorized, and recovered in the event of failures to ensure data integrity across both databases.

3.10.1 Admin writes

Administrative operations, such as the creation or the update of an entity, are executed using strongly consistent transactional workflows. These actions write to both MongoDB and Neo4j in a tightly coupled manner, ensuring the update either **completes successfully** in both systems or is **rolled back entirely**. Admin writes bypass the outbox mechanism, as consistency is enforced immediately at the point of write.

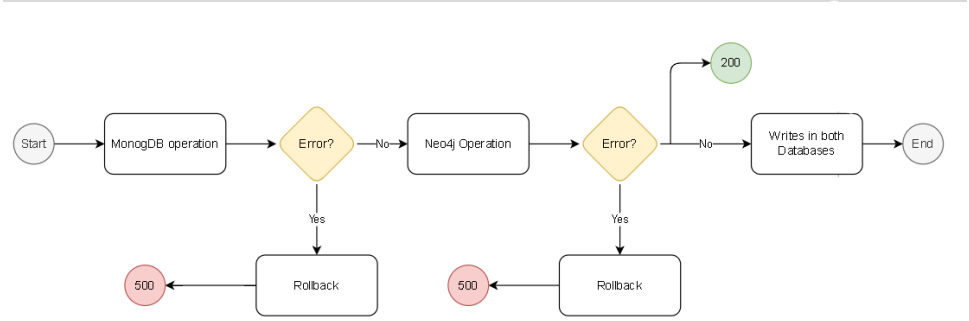

Figure 3.23: Admin Writes Workflow

3.10.2 Users writes

User-driven changes, like following other users, posting articles, or editing profiles, use an **asynchronous- eventually consistent** approach. These writes first persist to MongoDB, while corresponding synchronization events are queued in the outbox collection. Kafka ensures delivery to downstream services, and retries or

escalation to the failed-events log are triggered in case of Neo4j write failures. This separation ensures responsiveness for users while maintaining consistency across databases over time.

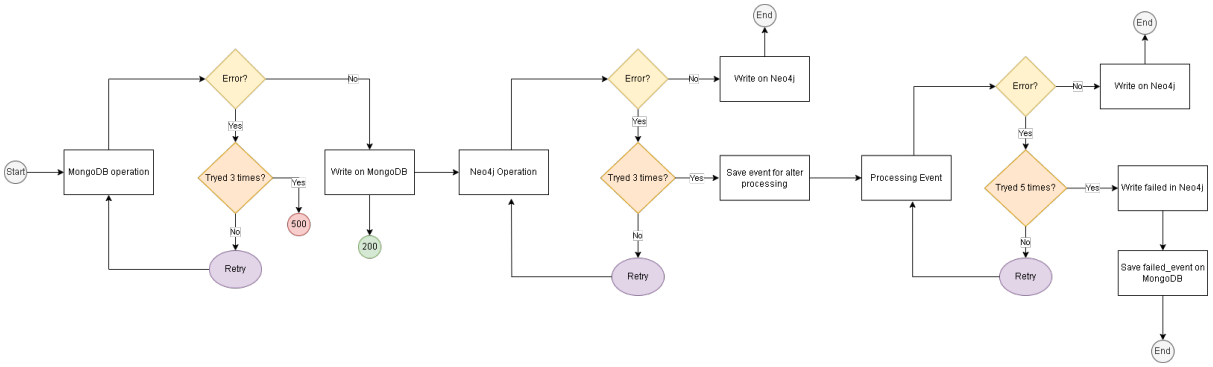

Figure 3.24: Users Writes Workflow

3.11 Distributed Database Design

Our distributed database structure is designed following an **AP-oriented** architecture, taking inspiration from the CAP theorem as a guiding principle. In this setup, we chose to prioritize Availability and Partition Tolerance, accepting the trade-off of potential temporary inconsistencies in favor of keeping the system always responsive and resilient to network partitions. This decision aligns with the nature of our application, where uninterrupted access to data and fault tolerance are more critical than strict, immediate consistency.

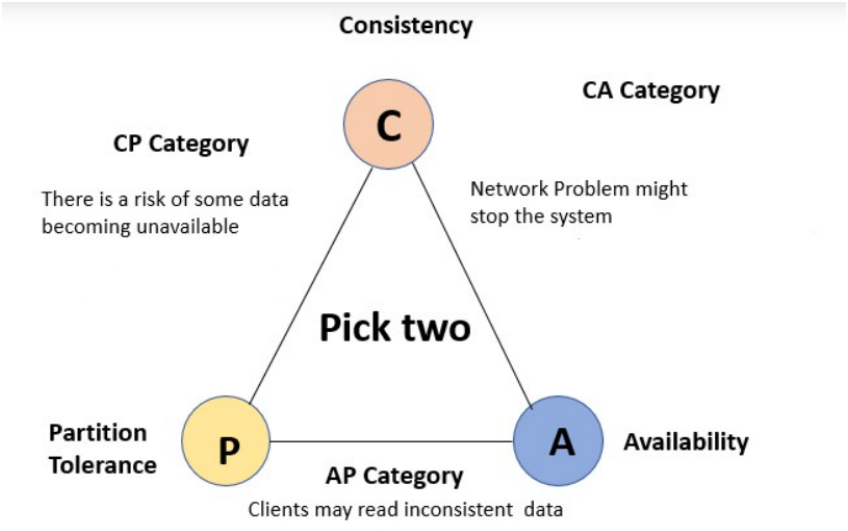

Figure 3.25: CAP Theorem

3.11.1 Replicas

A cluster of three nodes was made available for this project, enabling the deployment of a replica set architecture. Replication was implemented exclusively in MongoDB, since Neo4j requires the Enterprise edition to support clustering and

replication features. The infrastructure consisted of three Virtual Machines, each with a distinct IP address, which were accessed through a VPN setup. On each of the three nodes, MongoDB was installed and configured in replica set mode by launching the service with the appropriate –replSet parameter:

```
mongod--replSet lsmdb --dbpath /DATA --port
27020 --bind\_ip 10.1.1.* --oplogSize 200
```

After initiating each instance, the replica set configuration was completed using Mongo Shell commands, where node priorities and tags were assigned. These **priorities** were crucial: they determined the failover behavior, ensuring that if the primary node became unreachable, one of the remaining nodes would automatically be elected as the new primary based on the predefined priority rules.

```
cfg = rs.conf()
cfg.members[0].priority = 2
cfg.members[0].tags = { "region": "us-east", "rack": "rack1" }

cfg.members[1].priority = 3
cfg.members[1].tags = { "region": "us-east", "rack": "rack2" }

cfg.members[2].priority = 1
cfg.members[2].tags = { "region": "us-west", "rack": "rack3" }

rs.reconfig(cfg, { force: true })
```

Once the replica set was operational and stable, data was synchronized across all members, achieving redundancy and fault tolerance. This setup ensures that MongoDB can continue serving both read and write operations even in the presence of node failures

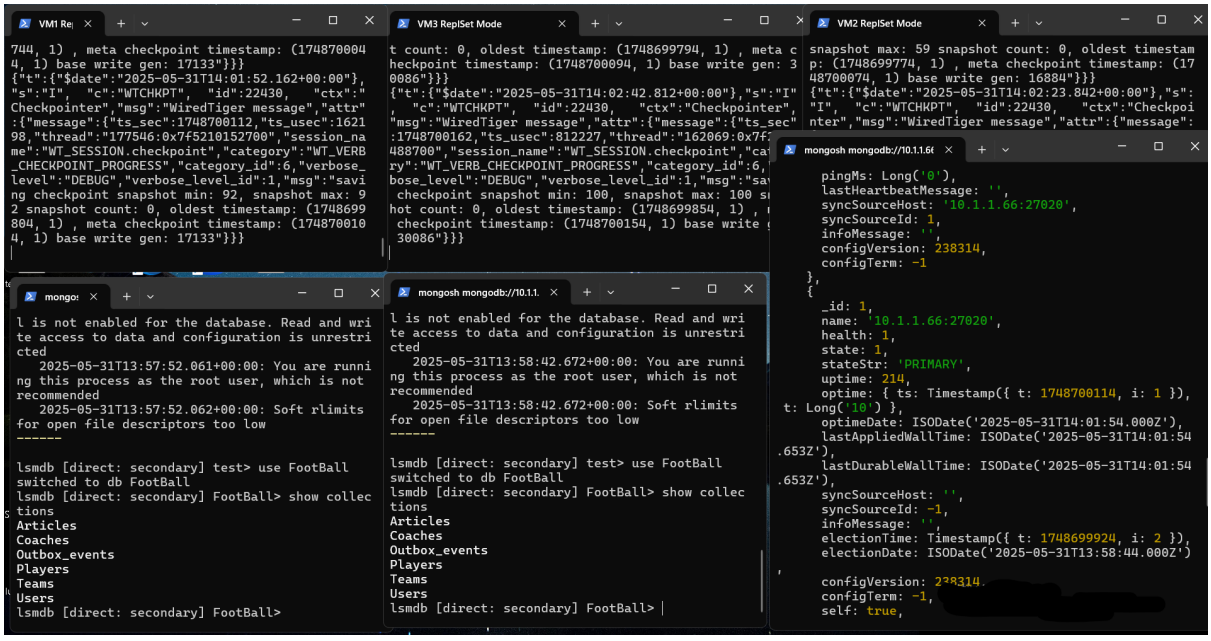

Figure 3.26: Replica Configurations

3.11.2 Concerns

In our distributed MongoDB deployment, we configured the system with write concern w:1 and read concern set to "nearest". These choices align with our AP-oriented architecture, prioritizing availability and partition tolerance over strict consistency, as suggested by the CAP theorem. Setting write concern to w:1 means that a write is **acknowledged** as soon as the primary node has received and written the data. This ensures low latency and high availability for write operations, even in the presence of network partitions or failures of secondaries. It allows us to maintain a responsive system under heavy load or when parts of the cluster are temporarily unreachable. For reads, the **primaryPreferred** setting gives us the best of both worlds: it reads from the primary when available (preserving stronger consistency), but gracefully falls back to secondaries during failovers or primary unavailability. This choice enhances fault tolerance and uptime, allowing users to access data even during transient disruptions. Following the SpringBoot integrated code to set up the replicas:

```
spring.data.mongodb.uri=mongodb://10.1.1.*:27020,10.1.1.*:27020,10.1.1.*:
27020/replicaSet=lsmdb&readPreference=primaryPreferred&w=1&wtimeoutMS=5000
```

3.11.3 Sharding

Although sharding was not implemented in the current deployment, the architecture was designed with a potential partitioned MongoDB cluster (consequently also Neo4j) in mind, especially to scale horizontally as data volume increases. Different options worth attention, considering that for this system the majority of the storage space, will be occupied by users and articles, one approach could be:

- Apply shards only on Users and Articles collections;
- Replicate all the other collections in each server of the cluster;
- Use as sharding keys , the **username** field of the user, and the **author** field of the article, so at the end each server will contain the user and its articles.

This approach exploits the difference of volumes in the collections, providing a compromise between availability and storage space management. Another option, a more common one, is :

- Sharding all the collections;
- Use as sharding keys , the **nationality_name** field when present (Users,Coaches, Players,Teams), and the **mongoId** field for articles.

This option, will partition the data, by geographical location, having less replicated data, but also less availability than the previous one.

Chapter 4

Implementation

4.1 Tools and Technologies

The implementation of the system leverages a range of modern, open-source technologies, each chosen for its suitability to different layers of the application stack:

- **Java 17** – Serves as the primary programming language for backend development, offering modern language features and long-term support.
- **Spring Boot 3.4.4** – A robust framework used to build and manage the RESTful API layer, simplifying dependency management and application configuration.
- **MongoDB** – A NoSQL document-oriented database used for storing core domain data with flexible schema design.
- **Neo4j** – A native graph database designed to manage and efficiently query complex relationship-based data.
- **Swagger UI** – Integrated for generating interactive API documentation, enabling testing and exploration of REST endpoints.
- **Maven 4.0** – Used for project build automation and managing dependencies across modules.
- **Docker** – Employed to containerize application components, ensuring consistent environments across development and production.
- **Apache Kafka 7.4.0** – A distributed event streaming platform used for handling asynchronous communication between services.
- **Apache ZooKeeper 3.8.0** – Provides coordination and configuration management for Kafka clusters.

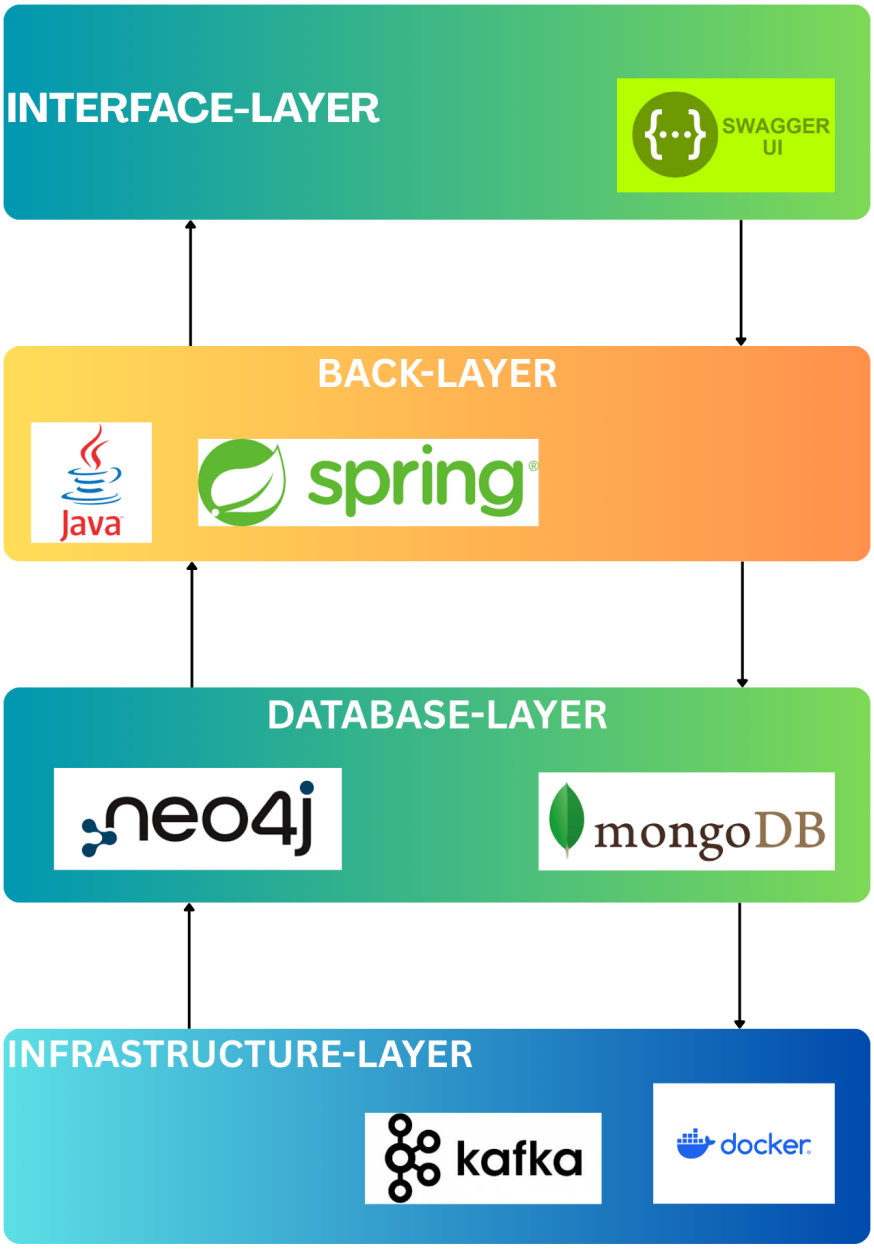

Figure 4.1: Software Architecture

4.2 Packages and Classes Organization

The architecture of the project is structured around a layered pattern that promotes modularity and clean separation of responsibilities. This design is implemented using Spring Boot and is divided into distinct layers, each serving a dedicated purpose within the system. At the core of this structure are four principal layers: Models, Repositories, Services, and Controllers.

Model layer is responsible for defining the application's data structures, which are directly mapped to database representations-whether MongoDB documents or Neo4j nodes. These model classes encapsulate the domain entities and reflect the real data managed by the application.

Repositories act as the abstraction layer between the application and the database. Using Spring Data, they simplify database operations through declarative method names that are automatically translated into queries. This abstraction removes the need for boilerplate code and ensures consistency across data access logic.

The service layer encapsulates the business logic of the application. Services are responsible for processing data, managing transactions, and applying any application-specific rules. They act as the intermediary between the repository and controller layers, ensuring a clean separation between business logic and presentation logic.

Controllers represent the interface between the user and the backend services. They receive HTTP requests, delegate operations to the appropriate services, and return responses.

In addition to the primary architectural layers, the project includes several supporting packages. These auxiliary modules are designed to handle specific technical or structural concerns, such as configuration classes, relationship mappings for graph database interactions, data transfer objects (DTOs), projections for partial queries, and Kafka integration for asynchronous event processing. This organized package structure not only enhances readability and maintainability but also ensures that the codebase remains scalable and extensible as the application evolves.

4.2.1 MongoDB

- */Football/src/java/.../demo/models/MongoDB*

  The MongoDB model package contains all entity classes annotated with `@Document`, mapping each class to its respective MongoDB collection. Nested objects within documents are represented by simple classes without annotations.

  1. **Articles.java**
  2. **Users.java**
  3. **Players.java**
  4. **Teams.java**

5. **Coaches.java**
6. **FifaStatsPlayer.java**
7. **FifaStatsTeam.java**
8. **TeamObj.java**
9. **CoachObj.java**
10. **ROLES.java**
11. **OutboxEvent.java**

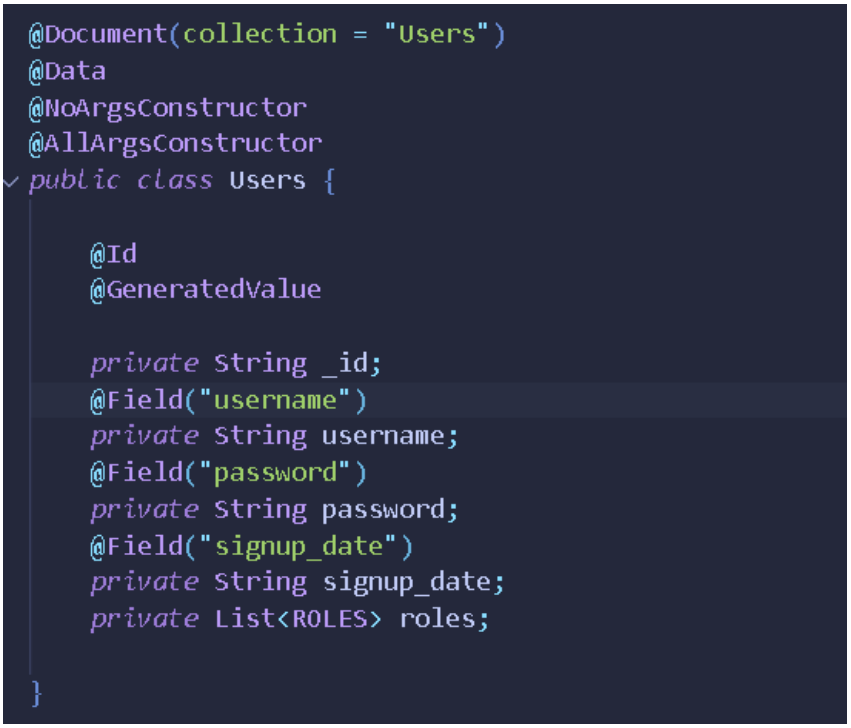

Figure 4.2: User model

- */Football/src/java/.../demo/repositories/MongoDB*

  MongoDB Repositories are interfaces annotated with `@Repository`, where query methods are declared using Spring Data conventions. These allow seamless access to MongoDB without manual query definitions.

  1. **Articles_repository.java**
  2. **Users_repository.java**
  3. **Players_repository.java**
  4. **Teams_repository.java**
  5. **Coaches_repository.java**
  6. **OutboxEvent_repository.java**

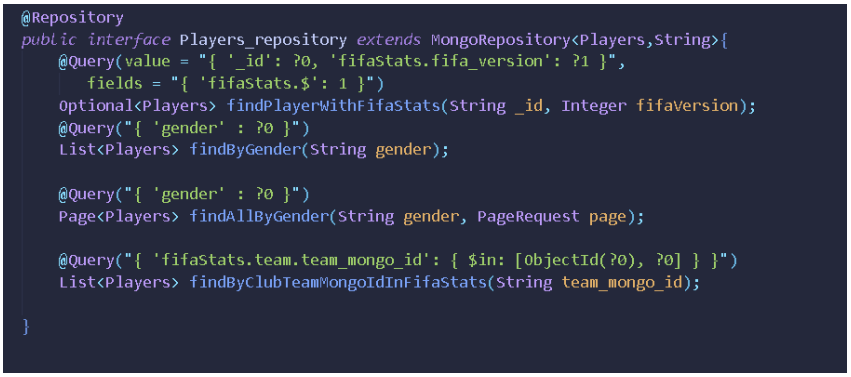

Figure 4.3: Players repository

- */Football/src/java/.../demo/services/MongoDB*

  MongoDB Service classes, annotated with `@Service`, implement the business logic and interact directly with repositories.

  1. **Articles_service.java**
  2. **Users_service.java**
  3. **Players_service.java**
  4. **Teams_service.java**
  5. **Coaches_service.java**
  6. **GlobalSearch_service.java**
  7. **MongoUserDetail_service.java**

```java
@Service
public class Articles_service {

    private final Articles_repository Ar;
    private final OutboxEvent_repository outboxEventRepository;
    private final ObjectMapper objectMapper;

    public Articles_service(Articles_repository ar,
    OutboxEvent_repository outboxEventRepository,
    ObjectMapper objectMapper) {
        this.Ar = ar;
        this.outboxEventRepository = outboxEventRepository;
        this.objectMapper = objectMapper;
    }

    //READ
    public Articles getArticle(String id){

        Optional<Articles> article = Ar.findById(id);
        if(article.isPresent()){
            return article.get();
        }
        else{
            return null;
        }
    }
```

Figure 4.4: Article service - Constructor and one method

- */Football/src/java/.../demo/controllers/MongoDB*

  MongoDB Controllers are the entry point for handling incoming HTTP requests. This package hosts all the controller classes that manage the interaction between the user and the application's services. These classes are annotated with **@RestController** and are responsible for returning HTML pages to the user,

  1. **Articles_controller.java**
  2. **Users_controller.java**
  3. **Players_controller.java**
  4. **Teams_controller.java**
  5. **Coaches_controller.java**
  6. **GlobalSearch_controller.java**
  7. **MongoUserDetail_controller.java**

```java
@RestController
@RequestMapping("/api/v1/")


public class Teams_controller {

    private final Teams_service teamsMService;

    public Teams_controller(Teams_service teamsMService) {
        this.teamsMService = teamsMService;
    }

    // READ: Get team by ID
    @GetMapping("team/{_id}")
    @Operation(summary = "READ: Get a team document by its ID", tags={"Team"})
    public Teams getTeam(@PathVariable String _id) {
        return teamsMService.getTeam(_id);
    }
```

Figure 4.5: Team controller - Constructor and one method

4.2.2 Neo4j

- */Football/src/java/.../demo/models/Neo4j*

The Neo4j packages mirror the MongoDB structure. Neo4j Model classes are annotated with `@Node` and represent the graph nodes.

1. **ArticlesNode.java**
2. **UsersNode.java**
3. **PlayersNode.java**
4. **TeamsNode.java**
5. **CoachesNode.java**

```java
@Node(labels = "ArticlesNode")
public class ArticlesNode {
    @Id
    @GeneratedValue
    private Long _id;

    @Property(name = "mongoId")
    @Indexed(unique = true)
    private String mongoId;
    @Property(name = "author")
    private String author;
    @Property(name = "title")
    private String title;

    @JsonIgnore
    @Relationship(type = "LIKES", direction = Relationship.Direction.INCOMING)
    private List<UsersNode> usersNode = new ArrayList<>();
    @JsonIgnore
    @Relationship(type = "WROTE", direction = Relationship.Direction.INCOMING)
    private List<UsersNode> wroteNode = new ArrayList<>();
}
```

Figure 4.6: ArticlesNode model

- */Football/src/java/.../demo/repositories/Neo4j* Neo4j Repositories use `@Repository` to define Cypher queries for node and relationship handling.

    1. **Articles_node_rep.java**
    2. **Users_node_rep.java**
    3. **Players_node_rep.java**
    4. **Teams_node_rep.java**
    5. **Coaches_node_rep.java**

```java
@Repository
public interface Coaches_node_rep extends Neo4jRepository<CoachesNode,Long>{

    boolean existsByMongoId(String valueOf);
    @Query("MATCH (c:CoachesNode {mongoId: $mongoId}) " +
           "RETURN c.mongoId AS mongoId, c.longName AS longName, c.gender AS gender")
    Optional<CoachesNodeDTO> findByMongoIdLight(String mongoId);

    Optional<CoachesNode> findByMongoId(String id);

    @Query(
        value = "MATCH (c:CoachesNode {gender: $gender}) " +
                "RETURN c.mongoId AS mongoId, c.longName AS longName, c.gender AS gender SKIP $skip LIMIT $limit",
        countQuery = "MATCH (c:CoachesNode {gender: $gender}) RETURN count(c)"
    )
    Page<CoachesNodeDTO> findAllByGenderWithPaginationLight(String gender, Pageable page);

    List<CoachesNode> findAllByGender(String gender);

    @Query("MATCH (c:CoachesNode) -[r:MANAGES_TEAM]-> (t:TeamsNode) " +
    "WHERE r.fifaVersion = $fifaV " +
    "AND t.mongoId = $mongoId " +
    "RETURN c.mongoId AS mongoId, c.longName AS long, c.gender AS gender, r.fifaVersion AS fifaVersion")
    Optional<CoachesNodeDTO> findFifaVersionByMongoIdAndFifaV(String mongoId, Integer fifaV);
```

Figure 4.7: CoachesNode repository

- */Football/src/java/.../demo/services/Neo4j*

    1. **Articles_node_service.java**
    2. **Users_node_service.java**
    3. **Players_node_service.java**
    4. **Teams_node_service.java**
    5. **Coaches_node_service.java**

- */Football/src/java/.../demo/controllers/Neo4j*

    1. **Articles_node_controller.java**
    2. **Users_node_controller.java**
    3. **Players_node_controller.java**
    4. **Teams_node_controller.java**
    5. **Coaches_node_controller.java**

4.2.3 Configurations

- */Football/src/java/.../demo/configurations/Neo4j*

The Neo4j Configurations package contains classes that are responsible for setting up and customizing the behavior of our Neo4j DB. This includes configurations for data sources, security, and any third-party integrations, i.e. Docker, Kafka and Zookeper. In here we have normal classes or classes with the annotation **@Configuration**.

1. **Neo4jConfig.java**:

   This Neo4jConfig class is a custom Spring configuration that manages how Spring Boot application connects to Neo4j in a **flexible** and **fault-tolerant** way. It sets up a custom Neo4j driver that doesn't connect directly, but instead uses a proxy *(DynamicDriverProxy)* to always delegate calls to the current active driver managed by **Neo4jDriverManager**; this allows the app to handle Neo4j being temporarily unavailable by centralizing connection logic and ensuring that all repository interactions go through a flexible, runtime-resolved driver.

```java
@Configuration
@Slf4j
@EnableNeo4jRepositories(basePackages = "com.example.demo.repositories.Neo4j")
public class Neo4jConfig {

    @Autowired
    private Neo4jDriverManager driverManager;

    @Bean
    public Driver neo4jDriver() {
        // Return a proxy that always delegates to the current driver
        return new DynamicDriverProxy(driverManager);
    }

    /**
     * Proxy that delegates to the current driver from DriverManager
     */
    private static class DynamicDriverProxy implements Driver {
        private final Neo4jDriverManager driverManager;

        public DynamicDriverProxy(Neo4jDriverManager driverManager) {
            this.driverManager = driverManager;
        }
```

Figure 4.8: Snippet of Neo4jConfig class

2. **Neo4jDriverManager.java**:

   This class manages the Neo4j database connection in a **resilient** way: it tries to connect on startup, and if Neo4j is down, it switches to a dummy *NoOpDriver* that avoids crashing the app; it keeps the driver thread-safe using locks, allows reconnection attempts later, and switches back to a real driver if the database becomes available again, ensuring smooth recovery from outages. This class is annoted as **@Component**.

```java
@Component
@Slf4j
public class Neo4jDriverManager {

    private final String uri;
    private final String username;
    private final String password;
    private final Config driverConfig;

    private volatile Driver currentDriver;
    private volatile boolean isRealDriver = false;
    private final ReadWriteLock driverLock = new ReentrantReadWriteLock();

    public Neo4jDriverManager(@Value("${spring.neo4j.uri}") String uri,
                              @Value("${spring.neo4j.authentication.username}") String username,
                              @Value("${spring.neo4j.authentication.password}") String password) {
        this.uri = uri;
        this.username = username;
        this.password = password;
        this.driverConfig = Config.builder()
                .withMaxConnectionLifetime(value:30, java.util.concurrent.TimeUnit.MINUTES)
                .withMaxConnectionPoolSize(value:50)
                .withConnectionAcquisitionTimeout(value:5, java.util.concurrent.TimeUnit.SECONDS)
                .build();

        // Initialize with either real driver or NoOp
        initializeDriver();
```

Figure 4.9: Neo4jDriverManager class - Constructor

```java
private void initializeDriver() {
    try {
        Driver realDriver = GraphDatabase.driver(uri, AuthTokens.basic(username, password), driverConfig);
        realDriver.verifyConnectivity();

        driverLock.writeLock().lock();
        try {
            this.currentDriver = realDriver;
            this.isRealDriver = true;
        } finally {
            driverLock.writeLock().unlock();
        }

        log.info(msg:" Connected to Neo4j successfully at startup");
    } catch (Exception e) {
        driverLock.writeLock().lock();
        try {
            this.currentDriver = new NoOpDriver();
            this.isRealDriver = false;
        } finally {
            driverLock.writeLock().unlock();
        }

        log.warn(format:" Neo4j is down at startup, using NoOpDriver: {}", e.getMessage());
    }
}
```

Figure 4.10: Neo4jDriverManager class - Initializer Method

3. **NoOpDriver.java**:

   The NoOpDriver is a fallback implementation of the Neo4j Driver interface that intentionally throws exceptions when any database operation is attempted, signaling that Neo4j is unavailable; it *allows the application to keep running without crashing and enables graceful error handling* until the real connection is restored.

```
class NoOpDriver implements Driver {
    @Override
    public Session session() {
        throw new org.neo4j.driver.exceptions.ServiceUnavailableException(message:"Neo4j is unavailable");
    }

    @Override
    public Session session(SessionConfig config) {
        throw new org.neo4j.driver.exceptions.ServiceUnavailableException(message:"Neo4j is unavailable");
    }
```

Figure 4.11: NoOpDriver class - some Methods

4. **Neo4jHealthChecker.java**:

   The Neo4jHealthChecker is a component responsible for monitoring the **health** of the Neo4j database. It integrates with *Neo4jDriverManager* to periodically test connectivity, attempt automatic reconnections if the database becomes unavailable, and switch to a *NoOpDriver* when necessary. It works in coordination with the dynamic driver proxy defined in *Neo4jConfig*, ensuring that the application can degrade gracefully during outages and recover transparently once Neo4j becomes reachable again.

```
@Component
@Slf4j
public class Neo4jHealthChecker {

    @Autowired
    private Neo4jDriverManager driverManager;

    @Autowired
    private ApplicationEventPublisher eventPublisher;

    private volatile boolean neo4jHealthy;
    private volatile boolean wasHealthy;
    private final AtomicLong lastCheckTime = new AtomicLong();
    private final AtomicReference<Instant> lastFailureTime = new AtomicReference<>();
    private final AtomicLong consecutiveFailures = new AtomicLong(initialValue:0);

    // Configuration constants
    private static final long HEALTH_CHECK_INTERVAL_MS = 10_000; // 10 seconds
    private static final long SCHEDULED_CHECK_INTERVAL_MS = 30_000; // 30 seconds
    private static final int MAX_RETRY_ATTEMPTS = 3;

    public Neo4jHealthChecker() {
        // Initialize based on whether we start with a real connection
        this.neo4jHealthy = false; // Will be set correctly on first health check
        this.wasHealthy = false;
    }
```

Figure 4.12: Neo4jHealthChecker class - Constructor

```java
*/
private boolean testConnection() {
    for (int attempt = 1; attempt <= MAX_RETRY_ATTEMPTS; attempt++) {
        try {
            return executeHealthQuery();
        } catch (ServiceUnavailableException | SessionExpiredException e) {
            log.warn(format:"Neo4j health check failed (attempt {}/{}): {}",
                attempt, MAX_RETRY_ATTEMPTS, e.getMessage());

            if (attempt == MAX_RETRY_ATTEMPTS) {
                recordFailure(e);
                // Switch to NoOpDriver on final failure
                driverManager.switchToNoOpDriver();
                return false;
            }

            // Brief pause between retries
            try {
                Thread.sleep(millis:1000);
            } catch (InterruptedException ie) {
                Thread.currentThread().interrupt();
                return false;
            }
        } catch (Exception e) {
            log.error(format:"Unexpected error during Neo4j health check: {}", e.getMessage(), e);
            recordFailure(e);
            driverManager.switchToNoOpDriver();
            return false;
        }
    }
    return false;
```

Figure 4.13: Neo4jHealthChecker class - One Method

5. **HealthStatus.java**:

   The HealthStatus class represents the health state of a Neo4j driver, tracking its current health, failure count, last failure time, and if a fallback driver is used. Used mainly to keep track of informations.

```java
public class HealthStatus {
        private final boolean healthy;
        private final long consecutiveFailures;
        private final Instant lastFailureTime;
        private final boolean usingNoOpDriver;

        public HealthStatus(boolean healthy, long consecutiveFailures,
                            Instant lastFailureTime, boolean usingNoOpDriver) {
            this.healthy = healthy;
            this.consecutiveFailures = consecutiveFailures;
            this.lastFailureTime = lastFailureTime;
            this.usingNoOpDriver = usingNoOpDriver;
        }

        public boolean isHealthy() { return healthy; }
        public long getConsecutiveFailures() { return consecutiveFailures; }
        public Instant getLastFailureTime() { return lastFailureTime; }
        public boolean isUsingNoOpDriver() { return usingNoOpDriver; }

        @Override
        public String toString() {
            return String.format(format:"HealthStatus{healthy=%s, failures=%d, usingNoOp=%s}",
                healthy, consecutiveFailures, usingNoOpDriver);
        }
    }
```

Figure 4.14: HealthStatus class

6. **Neo4jRecoveryEvent.java**:

   Neo4jRecoveryEvent class represents a custom Spring application event,

which is used to signal that a recovery process related to Neo4j has occurred. It extends *ApplicationEvent*, allowing it to be published and listened to within the Spring event system.

```java
public class Neo4jRecoveryEvent extends ApplicationEvent {
    private final long consecutiveFailures;

    public Neo4jRecoveryEvent() {
        this(consecutiveFailures:0L);
    }

    public Neo4jRecoveryEvent(long consecutiveFailures) {
        super(source:"Neo4jRecoveryEvent");
        this.consecutiveFailures = consecutiveFailures;
    }

    public long getConsecutiveFailures() {
        return consecutiveFailures;
    }
}
```

Figure 4.15: Neo4jRecoveryEvent class

- */Football/src/java/.../demo/configurations/Swagger*

The Swagger Configurations package contains classes that are responsible mainly to set up Swagger and Swagger UI visualization and to implement security protocols.

1. **SecurityConfig.java**:

   The SecurityConfig class configures Spring Security for the application using MongoDB for user details. It manages sessions only when required, and sets up authorization rules based on URL patterns and user roles, restricting access to admin and user-specific endpoints accordingly. Public endpoints like signup, search, and API documentation are left open, while authentication is enforced on sensitive paths. The class integrates a custom MongoDB-based user details service and configures HTTP Basic authentication with a custom realm. It also customizes logout behavior to invalidate sessions, clear authentication, and delete cookies.

Figure 4.16: SecurityConfig class - Snippet of the code

2. **SwaggerConfig.java**:

   SwaggerConfig class sets up Swagger (OpenAPI) documentation for the application and applies *HTTP Basic Authentication* globally. It registers a SecurityScheme named "basicAuth" using HTTP Basic as the authentication method and adds a global SecurityRequirement, ensuring that all documented API endpoints reflect this security scheme unless overridden. This allows Admins and Users to interact with secured endpoints directly from the Swagger UI by providing credentials.

3. **TagsConfiguration.java**:

   TagsConfiguration class defines global tags for Swagger (OpenAPI) documentation for the application. Each **@Tag** annotation labels a group of API endpoints with a descriptive name and purpose, helping organize and clarify the API structure in the Swagger UI.

- */Football/src/java/.../demo/configurations/Transaction*

1. **AsyncConfig.java**:

   AsyncConfig class enables and customizes asynchronous execution for the application. It defines a named Executor bean with a thread pool tailored for high concurrency, setting limits on thread count, queue capacity, and idle thread timeout. The executor also ensures tasks complete on shutdown and propagates the current **SecurityContext** to asynchronous threads using a custom *TaskDecorator*. This is critical for maintaining user authentication and authorization when executing secured logic in background tasks.

Figure 4.17: AsyncConfig class - snippet of a method

4.2.4 Others

- */Football/src/java/.../demo/DTO*

  This package simply includes all the **Data Transfer Object** used to project only the most important attribute values or information of the entities as JSON result of some read queries.

  1. **ArticlesNodeDTO.java**
  2. **UsersNodeDTO.java**
  3. **UsersNodeProjection.java**
  4. **MongoUserDTO.java**
  5. **PlayersNodeDTO.java**
  6. **TeamsNodeDTO.java**
  7. **CoachesNodeDTO.java**
  8. **ffCountDTO.java**
  9. **globalSearchResult.java**

- */Football/src/java/.../demo/requests*

  Requests package contains the fils in which are implemented classes to be passed as input into methods whenever writes operations are involved. They're made essentially from the corresponding models class carryng only the attributes are needed for the writes. This classes simplify the write operations making them easy to compile and avoids fails. Something

  1. **addPlayerToTeamRequest.java**

2. **changePasswordRequest.java**
3. **createArticleRequest.java**
4. **createCoachRequest.java**
5. **createPlayerRequest.java**
6. **createTeamRequest.java**
7. **createUserRequest.java**
8. **createArticleRequest.java**
9. **registerUserRequest.java**
10. **updateArticle.java**
11. **updateCoach.java**
12. **updatePlayer.java**
13. **updateTeam.java**
14. **updateFifaPlayer.java**
15. **updateFifaTeam.java**
16. **updateTeamCoach.java**
17. **updateTeamPlayer.java**
18. **updateCoachTeam.java**

- */Football/src/java/.../demo/relationships*

  Relationships package is used to define custom Neo4j relationships classes, mainly to declare attributes for the relationship. This classes are annotated with **@RelationshiProperties** and they needs a **@Property** annotation, for the attribute field and a **@TargetNode** for the arrival node declaration.

  1. **has_in_F_team.java**
  2. **has_in_M_team.java**
  3. **manages_team.java**
  4. **plays_in_team.java**

```java
@RelationshipProperties
@AllArgsConstructor
@NoArgsConstructor
public class has_in_M_team{
    @RelationshipId
    @GeneratedValue
    private Long id;

    @Property
    private Integer fifaVersion;

    @TargetNode
    private PlayersNode PMn;
```

Figure 4.18: has_in_M_team class

- */Football/src/java/.../demo/kafka* Kafka Package contains only 2 classes which implements the async behavior of the **OutBoxEvents** creation/reception and consumption. This classes are used to handle the eventual consistency when Neo4j is down, writing collections in MongoDB to store events that could not be attempted. These two classes works together with the *OutBoxEvent_repository* and *OutBoxEvent* model.

  1. **OutBoxEventPublisher.java**: This class periodically checks the database for events that haven't yet been published, batches them for efficiency, and sends them asynchronously to Kafka.

```java
private void processUnprocessedEvents() {
    boolean hasMoreEvents = true;
    int processedTotal = 0;

    while (hasMoreEvents) {
        // Recupera batch di eventi non pubblicati
        List<OutboxEvent> events = outboxEventRepository.findByPublishedFalseAndRetryCountLessThan(
            MAX_RETRY_COUNT,
            PageRequest.of(pageNumber:0, BATCH_SIZE, Sort.by(...properties:"createdAt").ascending())
        );

        if (events.isEmpty()) {
            hasMoreEvents = false;
            continue;
        }

        // Processa ogni evento nel batch
        for (OutboxEvent event : events) {
            processOutboxEvent(event);
            processedTotal++;
        }

        // Se il batch è pieno, potrebbe esserci ancora eventi
        hasMoreEvents = events.size() == BATCH_SIZE;
    }

    if (processedTotal > 0) {
        log.info(format:"Processed {} outbox events", processedTotal);
    }
```

Figure 4.19: OutBoxEventPublisher class - MongoDB Check Method

Each event is wrapped in a transactional context to ensure that it ei-

ther gets published and marked as such in the database, or its failure is recorded—updating the retry count and potentially marking it as permanently failed if retries exceed the configured maximum. The class listens for recovery signals (like Neo4j becoming available again) and triggers event reprocessing when appropriate.

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void processOutboxEvent(OutboxEvent event) {
    try {
        // Per eventi Neo4j, controlla se Neo4j è disponibile
        if (isNeo4jEvent(event.getEventType()) && !neo4jHealthChecker.isNeo4jHealthy()) {
            log.debug(format:"Neo4j is down, skipping event: {}", event.getEventType());
            return;
        }

        // Pubblica su Kafka
        publishEventToKafka(event)
            .thenAccept(result -> markEventAsPublished(event))
            .exceptionally(failure -> handleEventFailure(event, failure));

    } catch (Exception e) {
        log.error(format:"Exception processing outbox event {}: {}", event.get_id(), e.getMessage());
        handleEventFailure(event, e);
    }
}
```

Figure 4.20: OutBoxEventPublisher class - Check Neo4j and Publish event Method

Additionally, it includes scheduled tasks to retry failed events and clean up already published ones to keep the outbox table lean.

2. **ResilientEventConsumer.java**:

The ResilientEventConsumer listens to Kafka events (topic-based) and processes them. When an event is received, it first checks whether the Neo4j database is healthy. If it is, the consumer processes the event immediately by invoking the appropriate handler based on the event type. If Neo4j is down, it saves the event for later processing.

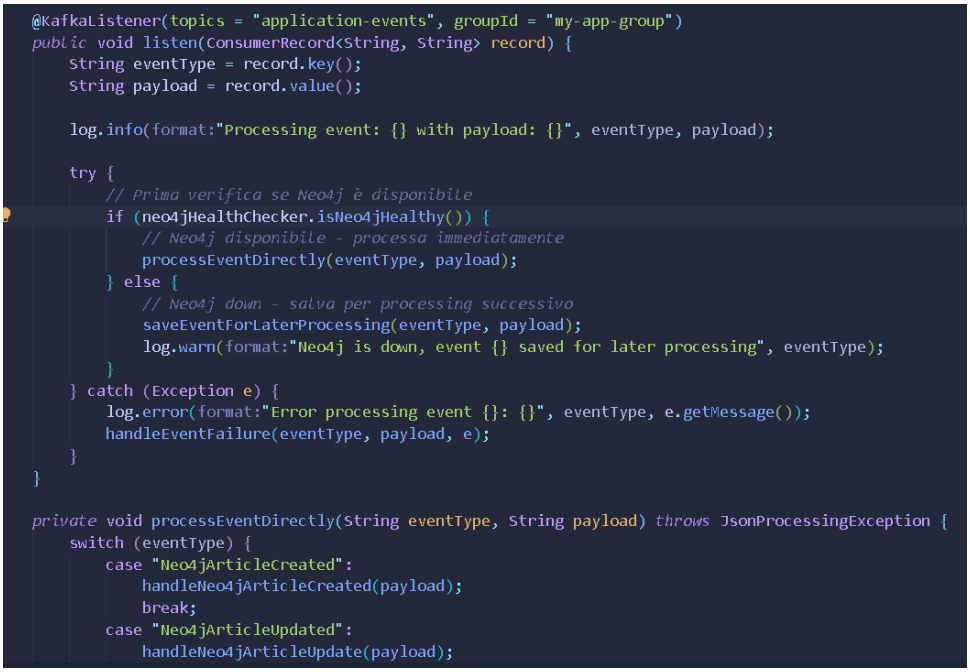

Figure 4.21: ResilientEvetConsumer class - Listener and Consumer methods

- *FootBall/aggregations/DTO* The aggregations DTO package contains all the classes implementing DTO to be used as results of the pipeline implemented, both in MongoDB and Neo4j.

  1. **DreamTeamPlayer.java**
  2. **faucetCountDTO.java**
  3. **facetResultDTO.java**
  4. **monthSummary.java**
  5. **MostEngaggedPlayer.java**
  6. **TeamImprovements.java**
  7. **TopTeam.java**
  8. **UserInterestDiversity.java**

- *FootBall/aggregations/MongoDB* MongoDB aggregation Package contains two files, a service and a controller which implements all the MongoDB aggregation method and relative controller.

  1. **FootBallService.java**
  2. **FootBallController.java**

- *FootBall/aggregations/Neo4j* Neo4j aggregation Package also contains two files, a service and a controller which implements all the Neo4j aggregation method and relative controller.

  1. **NodeService.java NodeController.java**

- *FootBall / data*

  Contains 4 Excel files used to populate the neo4j database. In each file can be found various combination of names used to create relationships bewteen nodes into the database.

- *FootBall / kafka-docker*

  It contains a .pml file in which can be found the configuration of Docker, Kafka and Zookeper.

- *FootBall / Mongosh*

  It contains a **DataSet_Manipulation.txt** file in which are reported all the pipeline used to de-construct and build the dataset from the original one. Can be found also an **Aggregations.txt** file in which are reported the most rilevan queries used in naiive MongoDB language (Javascript)

4.3 Relevant Operations

The core queries in our web application include standard CRUD operations for each entity, along with more complex aggregation pipelines used for data navigation, statistical computation, and analytical processing. A significant portion of the write operations-particularly those related to administrative tasks-are designed to *update both MongoDB and Neo4j databases in parallel*. These operations are typically marked with the `@Transactional` annotation to ensure strong consistency across systems.

On the other hand, user-initiated write operations prioritize eventual consistency. These are handled asynchronously through Kafka and are not annotated with `@Transactional`, allowing the system to remain responsive while updates are propagated reliably in the background.

4.3.1 CRUD Operations

- **Create Operation**

  The process of creating entities such as Players, Coaches, and Teams follows a unified structure, with access restricted to administrators. For illustration purposes, we will focus on the creation flow of a Player entity.

```
1       @Transactional
2     @Async("customAsyncExecutor")
3     public CompletableFuture<Players> createPlayer(
          createPlayerRequest request){
4         Players playerM = new Players();
5         playerM.setAge(request.getAge());
6         playerM.setDob(request.getDob());
7         playerM.setGender(request.getGender());
8         playerM.setHeight_cm(request.getHeight_cm());
9         playerM.setWeight_kg(request.getWeight_kg());
```

```
        playerM.setLong_name(request.getLong_name());
        playerM.setNationality_id(request.getNationality_id());
        playerM.setNationality_name(request.getNationality_name());
        playerM.setShort_name(request.getShort_name());
        PMr.save(playerM);
        PlayersNode playerNode = new PlayersNode();
        playerNode.setMongoId(playerM.get_id());
        playerNode.setLongName(playerM.getLong_name());
        playerNode.setNationalityName(playerM.getNationality_name()
            );
        playerNode.setGender(playerM.getGender());
        Pmr.save(playerNode);
        return CompletableFuture.completedFuture(playerM);
    };
```

Listing 4.1: Players service - Create Method

As shown in the figure, the creation of a Player is structured in three key steps. First, the system receives a request from the administrator containing the necessary attributes. Second, a new Players object is instantiated using the default constructor, which assigns a unique _id and fills in default values before saving the entity into MongoDB. Finally, a corresponding PlayersNode is created and stored in Neo4j with a minimal set of attributes, ensuring linkage across the data layers.

This operation is wrapped within a transactional boundary. If any part of the process fails whether writing to MongoDB or Neo4j the entire transaction is rolled back to prevent data inconsistency between the two databases. Since this flow handles both database writes simultaneously, there is no separate create method for PlayersNode (or similar entities) elsewhere in the Neo4j service.

On the other hand, creation operations in Neo4j that originate from user actions such as forming relationships or publishing articles are handled asynchronously through Kafka using the Outbox pattern. For example, the creation of an Article entity by a user is delegated to a Kafka event handler that eventually writes the corresponding node to Neo4j, ensuring decoupling and eventual consistency.

```
@Retryable(
    value = { Exception.class },
    maxAttempts = 3,
    backoff = @Backoff(delay = 1000, multiplier = 2)
)
public CompletableFuture<Articles> createArticle(String username,
    createArticleRequest request) throws JsonProcessingException {
    // 1. Fetch User from MongoDB
    Optional<Users> optionalUser = Ur.findByUsername(username);
    if (optionalUser.isEmpty()) {
        throw new RuntimeException("User not found with username: "
            + username);
    }
    Users existingUser = optionalUser.get();
    LocalDateTime currentDate =  LocalDateTime.now();
    DateTimeFormatter timeFormatter = DateTimeFormatter.ofPattern("
       dd/MM/yyyy");
```

```
    String publishTime = currentDate.format(timeFormatter);
    // 2. Save Article in MongoDB
    Articles newArticle = new Articles();
    newArticle.setAuthor(existingUser.getUsername());
    newArticle.setTitle(request.getTitle());
    newArticle.setContent(request.getContent());
    newArticle.setPublish_time(publishTime);

    Articles savedArticle = Ar.save(newArticle);

    // 3. Prepare Outbox event to create ArticleNode in Neo4j and
       link it to UsersNode
    Map<String, Object> neo4jPayload = new HashMap<>();
    neo4jPayload.put("articleId", savedArticle.get_id());
    neo4jPayload.put("title", savedArticle.getTitle());
    neo4jPayload.put("author", savedArticle.getAuthor());
    neo4jPayload.put("username", existingUser.getUsername()); // to
        find UsersNode in Neo4j

    OutboxEvent neo4jArticleCreatedEvent = new OutboxEvent();
    neo4jArticleCreatedEvent.setEventType("Neo4jArticleCreated");
    neo4jArticleCreatedEvent.setAggregateId(savedArticle.get_id());
    neo4jArticleCreatedEvent.setPayload(objectMapper.
       writeValueAsString(neo4jPayload));
    neo4jArticleCreatedEvent.setPublished(false);
    neo4jArticleCreatedEvent.setCreatedAt(LocalDateTime.now());

    outboxEventRepository.save(neo4jArticleCreatedEvent);

    // 4. Return saved article immediately
    return CompletableFuture.completedFuture(savedArticle);
}
```

Listing 4.2: Users model - Create Article Operation

The process begins with the creation of a new article entity, using the attributes provided in the request. After the article is successfully stored in MongoDB, an OutBoxEvent is generated. This event is labeled with the type **Neo4jArticleCreated** and contains the relevant payload, which includes the necessary data to reproduce the article in the Neo4j database. Once the event is saved, it is picked up by a dedicated Kafka event listener. The listener filters events by type and only proceeds if the specified event (Neo4jArticleCreated) matches and the Neo4j database is confirmed to be operational. When these conditions are met, the corresponding event-handling method is triggered.

```
    private void processEventDirectly(String eventType, String
       payload) throws JsonProcessingException {
        switch (eventType) {
            case "Neo4jArticleCreated":
                handleNeo4jArticleCreated(payload);
                break;
    // Some other code

@Retryable(value = {Exception.class}, maxAttempts = 3, backoff =
   @Backoff(delay = 2000))
    private void handleNeo4jArticleCreated(String payload) throws
       JsonProcessingException {
```

```
10          Map<String, Object> eventData = objectMapper.readValue(
                payload, new TypeReference<>() {});
11          String articleId = (String) eventData.get("articleId");
12          String username = (String) eventData.get("username");
13
14          try {
15              // Verifica che i prerequisiti esistano
16              Optional<UsersNodeDTO> userOptional =
                    usersNodeRepository.findByUserNameLight(username);
17              if (userOptional.isEmpty()) {
18                  log.error("User not found for article creation: {}"
                        , username);
19                  return;
20              }
21
22
23              Optional<ArticlesNodeDTO> existingArticle =
                    articlesNodeRepository.findByMongoIdLight(articleId)
                    ;
24              if (existingArticle.isPresent()) {
25                  log.info("Article {} already exists in Neo4j,
                        skipping creation", articleId);
26                  return;
27              }
28
29              // Crea l'articolo
30              ArticlesNode article = new ArticlesNode();
31              article.setMongoId(articleId);
32              article.setTitle((String) eventData.get("title"));
33              article.setAuthor(username);
34              articlesNodeRepository.save(article);
35
36              // Crea la relazione
37              usersNodeRepository.createWroteRelationToArticle(
                    username, articleId);
38
39              log.info("Successfully created article {} and relation
                    for user {}", articleId, username);
40
41          } catch (Exception e) {
42              log.error("Failed to create article {} for user {}: {}"
                    , articleId, username, e.getMessage());
43              throw e; // Per trigger del retry
44          }
45      }
```

Listing 4.3: ResilientEventConsumer - handleNeo4jArticleCreated

This method retrieves the event payload from MongoDB and executes the Neo4j-specific logic. It first ensures that the related UsersNode (representing the article's author) exists. Then, it creates the corresponding ArticlesNode in Neo4j and establishes the relationship between the user and the article, maintaining consistency across both data stores.

- **Read Operations**

Read operations in the application serve two main purposes.

The first purpose is to provide lightweight "preview" data of entities, such as Players, Coaches, Teams, or other Users. These operations are executed through Neo4j, where nodes are designed to store only essential information. This allows quick and efficient retrieval when a user is simply browsing or performing a search, without requiring the full dataset from MongoDB. Neo4j is therefore ideal for serving fast, relationship-oriented lookups and overviews. The second purpose is to offer detailed information about an entity when needed—for instance, viewing a full player profile. These operations are performed through MongoDB, where the complete document and all its attributes can be accessed. This dual-level approach balances performance and completeness by leveraging both databases for what they are best suited. For example, a typical read operation in Neo4j for previewing a player looks like this:

```
@Query("MATCH (u:UsersNode {userName: $username})<-[:FOLLOWS]-(f:
    UsersNode) "+
       "RETURN f { .userName, .mongoId } AS UsersNodeProjection")
    List<UsersNodeProjection> findFollowersByUserName(String
        username);
```

Listing 4.4: UsersNode Repository - findFollowers Query

```
public List<PlayersNodeDTO> ShowUserFPlayers(String username) {
        Optional<UsersNodeDTO> userNodeOpt = Unr.
            findByUserNameLight(username);
        if(userNodeOpt.isPresent()){
            return Unr.findHasInFTeamRelationshipsByUsername(
                username);
        }
        else{
            throw new IllegalArgumentException("User with id: " +
                username + " not found");
        }
    }
```

Listing 4.5: UsersNode Service - findFollowers method

The output will be a list of DTO, which have 4-5 attributes to just identify the entity. A more deeply read in MongoDB is :

```
@Query("{ 'gender' : ?0 }")
    Page<Players> findAllByGender(String gender, PageRequest page);
```

Listing 4.6: Players Repository - findAllByGender

```
public Page<Players> getAllPlayers(PageRequest page, String gender)
    {
        return PMr.findAllByGender(gender, page);
    }
```

Listing 4.7: Players Service - getAllPlayers

Dispite the first example, here are returned all the document of the players matching the gender attribute, where for each document there are more than 30 attributes.

- **Update Operations**

  Similar to the creation process, update operations are primarily restricted to administrators. These begin by modifying the data in MongoDB and, depending on the context, are mirrored in Neo4j to maintain consistency across both databases. This dual-write strategy ensures data coherence when changes are made by trusted users.

  A particularly important update scenario involves the modification of the FifaStats object for entities like Teams, Players, and Coaches. This functionality supports two main purposes: adding a new FIFA version record to an existing entity, or updating the attributes of a specific FIFA version already associated with that entity. In both cases, the system carefully updates the MongoDB document and adjusts or recreates the corresponding relationships and nodes in Neo4j to reflect the changes accurately. For istance we have:

```
@Transactional
@Async("customAsyncExecutor")
public CompletableFuture<Players> updateFifaPlayer(String id,
    Integer fifaV, updateFifaPlayer request) {
     Optional<Players> optionalPlayer = PMr.findById(id);
     if (optionalPlayer.isEmpty()) {
          throw new RuntimeException("Player not found with id: " +
             id);
     }

     Players existingPlayer = optionalPlayer.get();
     Optional<PlayersNodeDTO> optionalPlayerNode = Pmr.
        findByMongoIdLight(existingPlayer.get_id());
     if (optionalPlayerNode.isEmpty()) {
          throw new RuntimeException("Player with id: " + id + " not
             correctly mapped in Neo4j");
     }

     List<FifaStatsPlayer> stats = existingPlayer.getFifaStats();
     for (FifaStatsPlayer stat : stats) {
          if (!stat.getFifa_version().equals(fifaV)) continue;

          // If FIFA version changed, update relationships
          if (!fifaV.equals(request.getFifa_version())) {
               stat.setFifa_version(request.getFifa_version());

               Pmr.createPlaysInTeamRelationToTeam(id, stat.getTeam().
                  getTeam_mongo_id(), request.getFifa_version());
               Pmr.deletePlaysInTeamRelationToTeam(id, stat.getTeam().
                  getTeam_mongo_id(), fifaV);

               List<UsersNodeDTO> users = Unr.
                  findUsersByMongoIdAndFifaVersion(id, fifaV);
               if (users != null) {
                    for (UsersNodeDTO user : users) {
                         updateUserTeamRelations(user, existingPlayer.
                            getGender(), request.getFifa_version());
                    }
               }
          }
```

```
33
34              updateStatValues(stat, request);
35              break;
36          }
37
38          return CompletableFuture.completedFuture(PMr.save(
                existingPlayer));
39      }
40
41      private void updateUserTeamRelations(UsersNodeDTO user, String
            gender, Integer newFifaVersion) {
42          if ("male".equals(gender)) {
43              List<PlayersNodeDTO> players = Unr.
                    findHasInMTeamRelationshipsByUsername(user.getUserName()
                    );
44              if (players != null) {
45                  for (PlayersNodeDTO player : players) {
46                      Unr.deleteHasInMTeamRelation(user.getUserName(),
                            player.getMongoId());
47                      Unr.createHasInMTeamRelation(user.getUserName(),
                            player.getMongoId(), newFifaVersion);
48                  }
49              }
50          } else if ("female".equals(gender)) {
51              List<PlayersNodeDTO> fPlayers = Unr.
                    findHasInFTeamRelationshipsByUsername(user.getUserName()
                    );
52              if (fPlayers != null) {
53                  for (PlayersNodeDTO fPlayer : fPlayers) {
54                      Unr.deleteHasInFTeamRelation(user.getUserName(),
                            fPlayer.getMongoId());
55                      Unr.createHasInFTeamRelation(user.getUserName(),
                            fPlayer.getMongoId(), newFifaVersion);
56                  }
57              }
58          }
59      }
60
61      private void updateStatValues(FifaStatsPlayer stat,
            updateFifaPlayer request) {
62          stat.setOverall(request.getOverall());
63          stat.setPotential(request.getPotential());
64          stat.setValue_eur(request.getValue_eur());
65          stat.setWage_eur(request.getWage_eur());
66          stat.setClub_position(request.getClub_position());
67          stat.setClub_jersey_number(request.getClub_jersey_number());
68          stat.setClub_contract_valid_until_year(request.
                getClub_contract_valid_until_year());
69          stat.setLeague_name(request.getLeague_name());
70          stat.setLeague_level(request.getLeague_level());
71
72          // Physical and technical attributes
73          stat.setPace(request.getPace());
74          stat.setShooting(request.getShooting());
75          stat.setDribbling(request.getDribbling());
76          stat.setPassing(request.getPassing());
77          stat.setDefending(request.getDefending());
78          stat.setPhysic(request.getPhysic());
```

```
79
80      // Attacking
81      stat.setAttacking_crossing(request.getAttacking_crossing());
82      stat.setAttacking_finishing(request.getAttacking_finishing());
83      stat.setAttacking_heading_accuracy(request.
            getAttacking_heading_accuracy());
84      stat.setAttacking_short_passing(request.
            getAttacking_short_passing());
85      stat.setAttacking_volleys(request.getAttacking_volleys());
86
87      // Skill
88      stat.setSkill_dribbling(request.getSkill_dribbling());
89      stat.setSkill_curve(request.getSkill_curve());
90      stat.setSkill_fk_accuracy(request.getSkill_fk_accuracy());
91      stat.setSkill_long_passing(request.getSkill_long_passing());
92      stat.setSkill_ball_control(request.getSkill_ball_control());
93
94      // Movement
95      stat.setMovement_acceleration(request.getMovement_acceleration
            ());
96      stat.setMovement_agility(request.getMovement_agility());
97      stat.setMovement_reactions(request.getMovement_reactions());
98      stat.setMovement_balance(request.getMovement_balance());
99      stat.setMovement_sprint_speed(request.getMovement_sprint_speed
            ());
100
101     // Power
102     stat.setPower_shot_power(request.getPower_shot_power());
103     stat.setPower_jumping(request.getPower_jumping());
104     stat.setPower_stamina(request.getPower_stamina());
105     stat.setPower_strength(request.getPower_strength());
106     stat.setPower_long_shots(request.getPower_long_shots());
107
108     // Mentality
109     stat.setMentality_aggression(request.getMentality_aggression())
            ;
110     stat.setMentality_interceptions(request.
            getMentality_interceptions());
111     stat.setMentality_positioning(request.getMentality_positioning
            ());
112     stat.setMentality_vision(request.getMentality_vision());
113     stat.setMentality_penalties(request.getMentality_penalties());
114
115     // Defending
116     stat.setDefending_marking_awareness(request.
            getDefending_marking_awareness());
117     stat.setDefending_standing_tackle(request.
            getDefending_standing_tackle());
118     stat.setDefending_sliding_tackle(request.
            getDefending_sliding_tackle());
119
120     // Goalkeeping
121     stat.setGoalkeeping_diving(request.getGoalkeeping_diving());
122     stat.setGoalkeeping_handling(request.getGoalkeeping_handling())
            ;
123     stat.setGoalkeeping_kicking(request.getGoalkeeping_kicking());
124     stat.setGoalkeeping_positioning(request.
            getGoalkeeping_positioning());
```

```
125     stat.setGoalkeeping_reflexes(request.getGoalkeeping_reflexes())
            ;
126 }
```

Listing 4.8: Players Service - findAllByGender

The update operation requires the administrator to provide the player's _id, the FIFA version to be modified, and a request object containing the updated values. Initially, the code retrieves the corresponding player document from MongoDB and attempts to fetch the associated node in Neo4j. If either lookup fails, an exception is raised.

Once both entities are found, the method searches for the specific **FifaStatsPlayer** object that matches the given FIFA version. If the version is being changed, existing relationships in Neo4j referencing the old version are removed and new ones are created for the updated version. The player's attributes—such as technical skills, physical attributes, and club details—are then overwritten with the values from the update request.

This also involves updating all related relationships that carry the "fifa version" as a property. If the player had no initial FIFA version (e.g., -1 as a default value), the system uses a MERGE clause when interacting with Neo4j to automatically create the new relationship rather than update an existing one. This ensures that even uninitialized entries are properly integrated into the graph structure.
On the other hand we also have updates writes only in MongoDB and are User-reserved. Here it is an example:

```
1     @Retryable(
2         value = { Exception.class },
3         maxAttempts = 3,
4         backoff = @Backoff(delay = 1000, multiplier = 2)
5     )
6     public CompletableFuture<String> changePassword(String username
          ,String oldPassword, String newPassword){
7         Optional<Users> optional = Ur.findByUsername(username);
8         if(optional.isPresent()){
9             Users existing = optional.get();
10            if(passwordEncoder.matches(oldPassword, existing.
                  getPassword())){
11                existing.setPassword(passwordEncoder.encode(
                      newPassword));
12                Ur.save(existing);
13                return CompletableFuture.completedFuture("Password
                      Changed Correctly!");
14            }
15            else{
16                throw new RuntimeErrorException(null, "Wrong
                      Password, try again!");
17            }
18        }
19        else{
20            throw new RuntimeErrorException(null, "User not found
                  in MongoDB");
21        }
22    }
```

Listing 4.9: User Service - Change Password

The method itself is asynchronous, returning a *CompletableFuture<String>*, allowing it to run without blocking the main execution thread. It begins by attempting to retrieve the user by their username from MongoDB. If the user exists, it verifies the old password using a password encoder. If the match is successful, it encodes the new password and saves the updated user object.

- **Delete Operations**

Delete operations in our web application are also designed to reflect the architectural balance between transactional consistency and asynchronous propagation. Since the system integrates MongoDB for core data and Neo4j for relationship modeling, deletions must be handled carefully to maintain coherence across both databases. When an administrator initiates a delete request, the operation is executed as a transactional sequence. The entity is removed from MongoDB, and the corresponding node and its relationships in Neo4j are also deleted. This process is typically encapsulated within a method annotated with **@Transactional**. In the Users view indeed, deletes operations even in this case can be eventually consistent. For istance we have those two example working in the same way as for the creation:

```
@Transactional
    @Async("customAsyncExecutor")
    public void deletePlayer(String id){
        Optional<Players> player = PMr.findById(id);
        Optional<PlayersNodeDTO> playerNode = Pmr.
           findByMongoIdLight(id);
        if(player.isPresent()){
            PMr.deleteById(id);
        }
        else{
            throw new RuntimeErrorException(null, "Player not found
                with id: " + id);
        }
        if(playerNode.isPresent()){
            //removing relationship in Neo4j
            PMs.deletePlayer(id);
        }
        else{
            throw new RuntimeErrorException(null, "Player not found
                with id: " + id);
        }

    }
------------That uses---------------------
@Query("MATCH (p:PlayersNode {mongoId: $mongoId}) " +
    "DETACH DELETE p")
    void deletePlayerByMongoIdLight(String mongoId);
```

Listing 4.10: Player Service - Delete Method

```
@Retryable(
```

```
        value = { Exception.class },
        maxAttempts = 3,
        backoff = @Backoff(delay = 1000, multiplier = 2)
    )
    public CompletableFuture<String> deleteArticle(String username,
        String articleId ) throws JsonProcessingException{
        Optional<Users> optionalUser = Ur.findByUsername(username);
        Optional<Articles> optionalArticle = Ar.findById(articleId)
            ;
        if(optionalUser.isPresent() && optionalArticle.isPresent())
            {
            Articles existingArticle = optionalArticle.get();
            Users existingUser = optionalUser.get();
            if(existingArticle.getAuthor().equals(existingUser.
                getUsername())){
                Ar.deleteById(articleId);
                //prepare Outbox event to delete ArticleNode in
                    Neo4j
                Map<String, Object> neo4jPayload = new HashMap<>();
                neo4jPayload.put("articleId", existingArticle.
                    get_id());
                neo4jPayload.put("username", username);
                OutboxEvent neo4jArticleDeleteEvent = new
                    OutboxEvent();
                neo4jArticleDeleteEvent.setEventType("
                    Neo4jArticleDeleted");
                neo4jArticleDeleteEvent.setAggregateId(articleId);
                neo4jArticleDeleteEvent.setPayload(objectMapper.
                    writeValueAsString(neo4jPayload));
                neo4jArticleDeleteEvent.setPublished(false);
                neo4jArticleDeleteEvent.setCreatedAt(LocalDateTime.
                    now());
                outboxEventRepository.save(neo4jArticleDeleteEvent)
                    ;
                return CompletableFuture.completedFuture("Request
                    submitted...");
            }
            else{
                throw new RuntimeErrorException(null, "User not
                    found with username: " + username+" or Article
                    not present with id:" + articleId);
            }
        }
        else{
            throw new RuntimeErrorException(null, "User not found
                with username: " + username+" or Article not present
                 with id:" + articleId);
        }
    }
```

Listing 4.11: Users Service - Delete Article

4.3.2 Analytics and Aggregations

1. **Global Search**

The Global Search feature is one of the most dynamic and user-centric queries in the application. Inspired by the search bars commonly found in social media platforms, it allows users to input any keyword and receive results that span across various types of entities such as users, players, coaches, and teams. Is also possible to specify a filter, to indicate which collection consider and what to exclude.

```
@Service
public class GlobalSearch_service {

    @Autowired
    private MongoTemplate mongoTemplate;
    private static final Map<String, String> collectionNameMappings;
    private static final Map<String, String> collectionAttributeMappings;

    static {
        collectionAttributeMappings = new HashMap<>(); // Initialize the HashMap
        collectionAttributeMappings.put("user","username");
        collectionAttributeMappings.put("team","team_name");
        collectionAttributeMappings.put("coach", "long_name");
        collectionAttributeMappings.put("player","long_name");

        collectionNameMappings = new HashMap<>(); // Initialize the HashMap
        collectionNameMappings.put("user","Users");
        collectionNameMappings.put("team","Teams");
        collectionNameMappings.put("coach", "Coaches");
        collectionNameMappings.put("player","Players");
    }
  public Page<globalSearchResult> globalSearch(String keyword, Pageable pageable, List<String> searchInCollections) {

        Pattern pattern = Pattern.compile(keyword, Pattern.CASE_INSENSITIVE);

        if(searchInCollections.isEmpty())
            searchInCollections.addAll(Arrays.asList("user", "coach", "team", "player"));


        List<Document> pipeline = new ArrayList<>();
        ListIterator<String> iterator = searchInCollections.listIterator();

        String collection=iterator.next();


        pipeline.add(new Document("$match",
        new Document(collectionAttributeMappings.get(collection), pattern)));
        pipeline.add(new Document("$project",
        new Document("mongo_id", "$_id")
                .append("name", "$"+collectionAttributeMappings.get(collection))
```

```
                .append("type", collection)));

        // Conditional $unionWith stages
        while(iterator.hasNext()){
            collection=iterator.next();
            pipeline.add(new Document("$unionWith",
            new Document("coll", collectionNameMappings.get(
                collection))
                    .append("pipeline", List.of(
                            new Document("$match",
                            new Document(
                                collectionAttributeMappings.get(
                                collection), pattern)),
                            new Document("$project",
                            new Document("mongo_id", "$_id")
                                    .append("name", "$"+
                                        collectionAttributeMappings.
                                        get(collection))
                                    .append("type", collection))
                    ))
            ));
        }

        if (pipeline.isEmpty()) {
            return new PageImpl<>(Collections.emptyList(), pageable
                , 0L);
        }


        // $facet for pagination and count
        Document facetStage = new Document("$facet", new Document()
                .append("data", List.of(
                        new Document("$skip", pageable.getOffset())
                            ,
                        new Document("$limit", pageable.getPageSize
                            ())
                ))
                .append("count", List.of(
                        new Document("$count", "count")
                ))
        );
        pipeline.add(facetStage);

        // Run aggregation
        Aggregation aggregation = Aggregation.newAggregation(
            pipeline.stream()
                .map(stage -> (AggregationOperation) context ->
                    stage)
                .toList());


        AggregationResults<facetResultDTO> results = mongoTemplate.
            aggregate(aggregation,
        collectionNameMappings.get(collection),
        facetResultDTO.class);
        facetResultDTO facetResult = results.getUniqueMappedResult
            ();

```

```
87          List<globalSearchResult> content = facetResult != null &&
                facetResult.getData() != null
88                  ? facetResult.getData()
89                  : Collections.emptyList();
90
91          long total = (facetResult != null && facetResult.getCount()
                != null &&
92           !facetResult.getCount().isEmpty())
93                  ? facetResult.getCount().get(0).getCount()
94                  : 0L;
95
96          return new PageImpl<>(content, pageable, total);
97      }
```

Listing 4.12: Global Search

```
{"$match": {"username": {"$regularExpression": {"pattern": "messi",
"options": "i"}}}}

 {"$project": {"mongo_id": "$_id", "name": "$username", "type": "user"}}

 {"$unionWith": {"coll": "Coaches", "pipeline": [
 {"$match": {"long_name": {"$regularExpression":
 {"pattern": "messi", "options": "i"}}}},
 {"$project": {"mongo_id": "$_id", "name": "$long_name", "type": "coach"}}]}}

 {"$unionWith": {"coll": "Teams", "pipeline": [
 {"$match": {"team_name": {"$regularExpression":
 {"pattern": "messi", "options": "i"}}}},
 {"$project": {"mongo_id": "$_id", "name": "$team_name", "type": "team"}}]}}

 {"$unionWith":
 {"coll": "Players", "pipeline": [
 {"$match": {"long_name": {"$regularExpression":
 {"pattern": "messi", "options": "i"}}}},
 {"$project": {"mongo_id": "$_id", "name": "$long_name", "type": "player"}}
 ]}}
 {"$facet":
 {"data": [{"$skip": 0}, {"$limit": 10}], "count":
 [{"$count": "count"}]}}
```

This aggregation performs a search on a single or across multiple collections, Users, Coaches, Teams, and Players, by unifying queries with **$unionWith** and applying a consistent projection structure. It begins by searching the first entity specified in the filter (if no filter provided it starts from Users collection) matching “messi” (case-insensitive), projecting each result with a standardized ID, name field, and a tag indicating the collection type. It then chains further matches into the following collections, each using similar logic to match on relevant fields like long_name or team_name, ensuring uniform

output. Finally, it wraps the results in a **$facet** stage that paginates the data and returns a count of total matches, allowing the client to handle both display and navigation of search results efficiently in a single query.

```
public class searchController {
    public searchController(GlobalSearch_service gss){
        this.gss=gss;
    }

    @GetMapping({"/{name}","/{name}/{filter}"})
    @Operation(summary = "Global search")
    public ResponseEntity<Page<globalSearchResult>> search(@PathVariable String name,Pageable pageable,
            @PathVariable(required = false) String filter) {
        List<String> allowedFilters = Arrays.asList(...a:"user", "coach", "team", "player");


        List<String> collectionsToSearch = null;
        if (filter == null){
            collectionsToSearch= Arrays.asList(...a:"user", "coach", "team", "player");
            //System.out.println(collectionsToSearch);

        }
        if (filter != null && !filter.trim().isEmpty()) {
            collectionsToSearch = Arrays.stream(filter.split(regex:","))
            .map(String::trim)
            .filter(s -> !s.isEmpty())
            .map(String::toLowerCase)
            .collect(Collectors.toList());

            collectionsToSearch.removeIf(s -> !allowedFilters.contains(s));
            //System.out.println(collectionsToSearch);
        }

        return ResponseEntity.ok(gss.globalSearch(name, pageable,collectionsToSearch));
    }

}
```

Figure 4.22: Global Search Controller

The controllers then powers the global search feature, with a particular focus on dynamic **filtering** based on entity types. When a search request is made, the controller accepts an optional filter parameter that lets clients specify which categories, such as user, coach, team, or player—they want to include in the search. The filter is flexible: it can contain one or more comma-separated values and is case-insensitive, the format is <collection,collection,...>. Before executing the search, the controller sanitizes and validates the filter values against a predefined list of allowed types, discarding any invalid entries. If no filter is provided, by default search across all the collections. This approach ensures that clients can precisely tailor their search scope without compromising data search performance.

2. **User Interest Diversity**

To support analytical capabilities within the application, one of the key aggregation queries developed focuses on evaluating user interest diversity. This metric is particularly useful when analyzing user behavior and interaction patterns within the platform. It helps to identify which users are engaging with a broader range of content types, be it through liking articles, interacting with teams, or following players. Such insights are valuable in tailoring recommendations, driving personalization strategies, or simply understanding usage trends at a glance.

On the other hand, we also designed pipelines to analyze and rank users based on the diversity of their interests. This query is implemented in Neo4j using Cypher, and operates by traversing three primary relationship types originating from user nodes: **LIKES**, **HAS_IN_M_TEAM**, and **HAS_IN_F_TEAM**.

```
1  public Collection<UserInterestDiversity>
       findUserInterestDiversity() {
2      return this.neo4jClient.query("MATCH (u:UsersNode)-[r:LIKES
           |HAS_IN_M_TEAM|HAS_IN_F_TEAM]->(entity) " +
3          "WITH u, " +
4          "COUNT(DISTINCT CASE WHEN TYPE(r) = 'LIKES' THEN entity
               END) AS likes, " +
5          "COUNT(DISTINCT CASE WHEN TYPE(r) = 'HAS_IN_M_TEAM'
               THEN entity END) AS hasInMTeamCount, " +
6          "COUNT(DISTINCT CASE WHEN TYPE(r) = 'HAS_IN_F_TEAM'
               THEN entity END) AS hasInFTeamCount " +
7          "WITH u, " +
8          "(likes + hasInMTeamCount + hasInFTeamCount) / 3.0 AS
               avgInterestCount " +
9          "RETURN u.userName AS userName, avgInterestCount " +
10         "ORDER BY avgInterestCount DESC " +
11         "LIMIT 10")
12         .fetchAs(UserInterestDiversity.class)
13         .mappedBy((typeSystem, record) -> {
14             String userName = record.get("userName").asString()
                   ;
15             Double avgInterestCount = record.get("
                   avgInterestCount").asDouble();
16             return new UserInterestDiversity(userName,
                   avgInterestCount);
17         })
18         .all();
19 }
```

Listing 4.13: UserInterestDiversity

Each of these relationships represents a different form of engagement: a user liking an article or a Coach, having a male player, or a female player, in its own team respectively, and so on. For every user, the query counts how many unique entities they are connected to via each of these three relationship types. These individual counts are then averaged to produce a single metric that quantifies the breadth of a user's interests. Once calculated, users are sorted by this average in descending order, and only the top 10 are returned. This allows the application to identify the most engaged or widely interested users across various dimensions of the system.

4.4 Tests

Some test about common queries were conducted, in order to asses the functionality of the system itself and also to retrieve some data about the performances. The equipment used is:

- MSI Katana GF66 12UD (Laptop) (12th Gen Intel(R) Core(TM) i7-12700H 2.70 GHz, 16GB 3200 MHz RAM, 4GB NVidia GForce RTX 3050Ti);

- Swagger UI.

The method used, is exploiting the response time option provided by Swagger UI to measure the response time of the queries.The measures provided have a confidence Interval of 95%. Is important to mention that this response time is related to the localhost, so they must be considered with nearly zero delay, a real implementation over internet, will add to these measurement additional delay.

4.4.1 Read operation

The operations considered are the global search (carried out on MongoDB) and retrieving the followers of a user (carried out in NEO4j), it was also considered the difference of having or not indexes, and how they influence those operations.

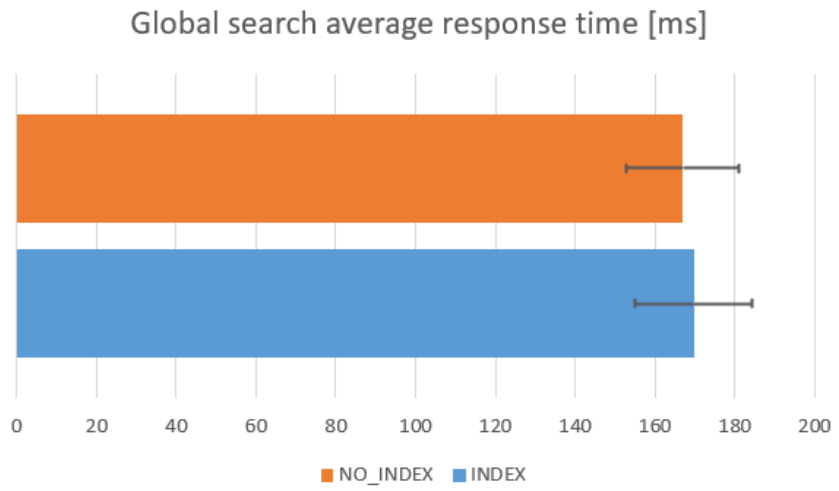

Figure 4.23: Global search test

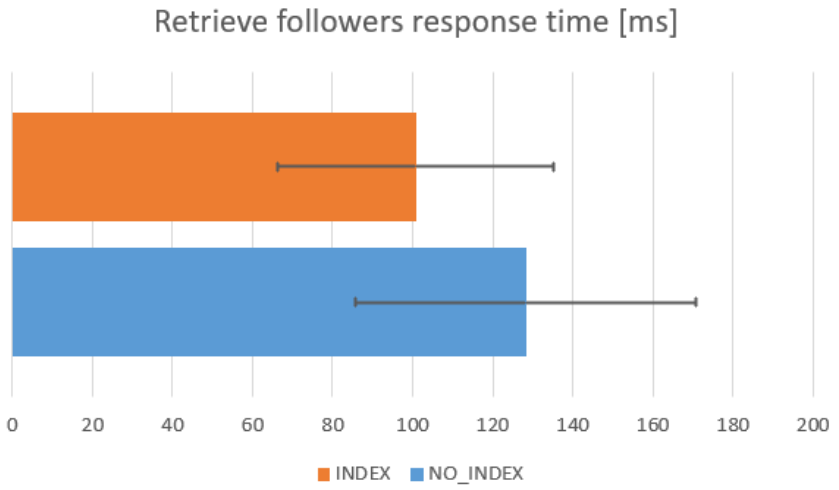

Figure 4.24: Retrieve followers test

For the global search the average response time was 170 ms, with indexes and 169 without, this is due the reasons provided in the section 3.9, which led to the conclusion of not using indexes. Regarding the retrieving of followers with index

the average response time was 101 ms and 128 ms without, the reason of such small improvement can be related to the small amount of users (around 340). This difference will probably increasing with a larger amount of users.

4.4.2 Write operations

The operations considered are a posting an article, and liking an article as shown in the plots 4.25, 4.26, both operations are carried out quite fast, the average response time for posting an article with 100 words is 164 ms,regarding liking an article the average response time is 120 ms

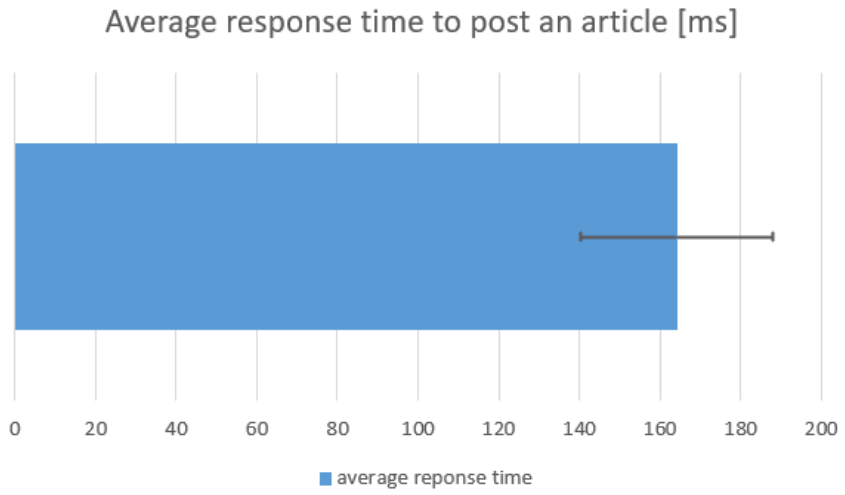

Figure 4.25: Post article test

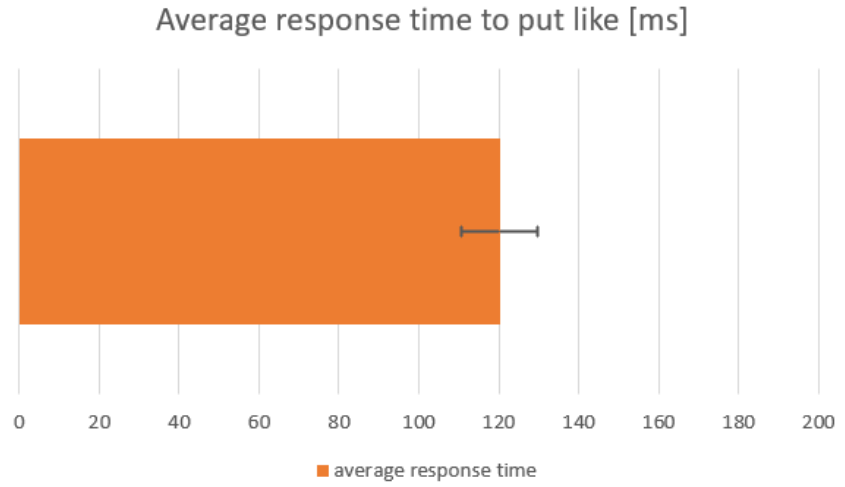

Figure 4.26: Like article test

4.5 API Documentation

The subsequent sections of this chapter provide detailed documentation of API endpoints.

4.5.1 API Endpoints: Articles Controller

1. **GET** `admin/articles`
   - **Inputs:**
     - `page` (optional): Integer specifying the page number (default is 0).
     - `size` (optional): Integer specifying how many articles per page (default is 20).
   - **Outputs:** Returns a JSON object containing an array of full content articles
2. **GET** `/articles/{_id}`
   - **Inputs:**
     - `_id`: String representing the unique MongoDB identifier of the article.
   - **Outputs:** Detailed JSON containing the full content of the article
3. **DELETE** `admin/articles/delete/{_id}`
   - **Inputs:**
     - `_id`: Unique identifier of the article to be deleted.
   - **Outputs:** HTTP status

4.5.2 API Endpoints: Coaches Controller

1. **GET** `/search/filter/coach/list/byGender/{gender}`
   - **Inputs:**
     - `page` (optional): Integer specifying the current page (default is 0).
     - `size` (optional): Integer specifying the number of coaches per page (default is 20).
     - `gender`: String specifying the grouping field
   - **Outputs:** Returns JSON containing a paginated list of coaches including their basic details grouped by gender
2. **GET** `/coaches/{_id}`
   - **Inputs:**
     - `_id`: Unique MongoDB identifier of the coach.
   - **Outputs:** Detailed JSON containing the coach's complete information
3. **POST** `/admin/coach/new`
   - **Inputs:** JSON object containing attributes to assign to the entity
   - **Outputs:** JSON response containing the newly created coach object along with its unique MongoDB ID.

4. **PUT** `/admin/coach/modify/{_id}`
    - **Inputs:**
        - `_id`: Unique identifier of the coach.
        - Request body with fields to update
    - **Outputs:** JSON containing the updated coach details.
5. **PUT** `/admin/coach/modify/{_id}/{oldFifa}/{newFifa}`
    - **Inputs:**
        - `_id`: Unique identifier of the coach.
        - `oldFifa`: Identifier to find fifa Version to Update
        - `newFifa`: New fifa Version to be uploaded
    - **Outputs:** JSON containing the updated coach details.
6. **PUT** `/admin/coach/modify/{_id}/{fifaV}`
    - **Inputs:**
        - `_id`: Unique identifier of the coach.
        - `fifaV`: Identifier to find fifa Version where the Team is stored
        - JSON object with attributes of the Team that must be updated
    - **Outputs:** JSON containing the updated coach details.
7. **DELETE** `/admin/coaches/{_id}`
    - **Inputs:**
        - `_id`: Unique identifier of the coach to delete.
    - **Outputs:** HTTP status
8. **DELETE** `/admin/coach/delete/team/{_id}/{fifaV}`
    - **Inputs:**
        - `_id`: Unique identifier of the coach to delete.
        - `fifaV`: Identifier to find fifa Version where the Team is stored
    - **Outputs:** HTTP status

4.5.3 API Endpoints: Players Controller

1. **GET** `/player/{_id}`
    - **Inputs:**
        - `_id`: Player's unique MongoDB ObjectId.
    - **Outputs:** JSON object containing details of the player.
2. **GET** `/admin/player/byGender/{gender}`
    - **Inputs:**

- `gender`: String value ("Male", "Female", etc.).
- `page` (optional): Page number for pagination.
- `size` (optional): Number of items per page.

- **Outputs:** Paginated list of players matching the gender criteria.

3. **POST** `/admin/player/new`
   - **Inputs:** JSON body containing player details
   - **Outputs:** JSON object representing the created player with the assigned ObjectId.
4. **PUT** `/admin/player/modify/{_id}`
   - **Inputs:**
     - `_id`: Player's unique identifier.
     - JSON request body with fields to update.
   - **Outputs:** JSON of updated player details.
5. **PUT** `/admin/player/modify/{_id}/{fifaV}`
   - **Inputs:**
     - `_id`: Player ID.
     - `fifaV`: Specific FIFA version number.
     - JSON request body with updated stats.
   - **Outputs:** Updated player object.
6. **PUT** `/admin/player/modify/{_id}/{fifaV}/team`
   - **Inputs:**
     - `_id`: Player ID.
     - `fifaV`: Specific FIFA version number where the Team is stored.
     - JSON request body with updated team stats.
   - **Outputs:** Updated player object.
7. **DELETE** `/admin/player/delete/{_id}`
   - **Inputs:**
     - `_id`: Player's MongoDB ObjectId.
   - **Outputs:** HTTP status

4.5.4 API Endpoints: Teams Controller

1. **GET** `/admin/team/byGender/{gender}`
   - **Inputs:**
     - `page` (optional): Page number for pagination.
     - `size` (optional): Number of teams per page.

- `gender`: Field name for grouping teams.
- **Outputs:** JSON response containing teams details grouped by gender.

2. **GET** `/teams/{_id}`
   - **Inputs:**
     - `_id`: MongoDB unique identifier of the team.
   - **Outputs**: JSON object containing details of the Team
3. **POST** `/admin/team/new`
   - **Inputs:** JSON payload with new team details.
   - **Outputs:** Newly created team JSON object including its MongoDB ID.
4. **PUT** `/admin/teams/modify/{_id}`
   - **Inputs:**
     - `_id`: Team's unique identifier.
     - JSON payload with fields to update.
   - **Outputs:** Updated team object.
5. **PUT** `/admin/team/modify/{_id}/{fifaV}`
   - **Inputs:**
     - `_id`: Team's unique identifier.
     - `fifaV`: Fifa version of the team to be updated
     - JSON payload with fields to update.
   - **Outputs:** Updated team object.
6. **PUT** `/admin/team/modify/{_id}/{fifaV}/coach`
   - **Inputs:**
     - `_id`: Team's unique identifier.
     - `fifaV`: Fifa version of the team to be updated
     - JSON payload with coach fields to update.
   - **Outputs:** Updated team object.
7. **DELETE** `/admin/teams/{_id}`
   - **Inputs:**
     - `_id`: Unique identifier of the team.
   - **Outputs:** HTTP status
8. **DELETE** `/admin/team/delete/{_id}/{fifaV}`
   - **Inputs:**
     - `_id`: Unique identifier of the team.
     - `fifaV`: Fifa version where the coach is stored
   - **Outputs:** HTTP status

4.5.5 API Endpoints: Users Controller

1. **GET** `/admin/users`
   - **Inputs:**
     - `page` (optional): Page number for pagination.
     - `size` (optional): Number of teams per page.
   - **Outputs:** JSON containing a list of users and relative details
2. **GET** `/admin/user/usernameSearch/{username}`
   - **Inputs:**
     - `username`: Attribute used to group Users
   - **Outputs:** JSON containing a list of users and relative details grouped by username
3. **POST** `/article/new`
   - **Inputs:** JSON body of the article to be created
   - **Outputs:** JSON response of the newly created article object, including its generated MongoDB identifier.
4. **POST** `/signup`
   - **Inputs:** JSON body of the user to be registered
   - **Outputs:** JSON response of the newly created user object, including its generated MongoDB identifier.
5. **POST** `/admin/create_admin`
   - **Inputs:** JSON body of the admin to be registered
   - **Outputs:** JSON response of the newly created user object, including its generated MongoDB identifier.
6. **PUT** `/user/article/edit/{articleId}`
   - **Inputs:**
     - `articleId`: Unique identifier for the targeted article.
     - Request body containing fields to update.
   - **Outputs:** JSON containing the updated article reflecting changes made.
7. **PUT** `user/modify/change_password`
   - **Inputs:**
     - JSON object with new password details
   - **Outputs:** HHTP status
8. **DELETE** `/admin/delete/{username}`

- **Inputs**:
  - `username`:User's unique username identifier.
- **Outputs:** HTTP status

9. **DELETE** `/user/settings/deleteAccount`
   - **Inputs**:
   - **Outputs:** HTTP status

4.5.6 API Endpoints: Global Search Controller

1. **GET** `/search/{name}`
   - **Inputs:**
     - `name`: Partial or full name of the entity.
   - **Outputs:** Paginated list of all objects that meet search criteria.
   - **Example**:
     (a) **Input**: Simone
     (b) **Output**:


```
                {
    "content": [
      {
        "name": "Simone Gambino",
        "mongo_id": "681f1e25e69dc7d7fad02719",
        "type": "user"
      },
      {
        "name": "Simone",
        "mongo_id": "681f1fb991358336b470ab72",
        "type": "user"
      },
      {
        "name": "Simone Inzaghi",
        "mongo_id": "6836e7471fea6d2694941e95",
        "type": "coach"
      },
      {
        "name": "Simone Charley",
        "mongo_id": "6836e6adae93aaefe539047a",
        "type": "player"
      },
      {
        "name": "Simone Melanie Laudehr",
        "mongo_id": "6836e6afae93aaefe539129a",
        "type": "player"
      }
```

2. **GET** `/search/{name}/{filter}`
    - **Inputs:**
        - `name`: Partial or full name of the entity.
        - `filter`: Filter used to group the results
    - **Outputs:** Paginated list of all objects that meet search criteria.

4.5.7 API Endpoints: Articles Node Controller

1. **GET** `admin/articleNode/articles`
    - **Inputs:**
        - `page` (optional): Page number for pagination.
        - `size` (optional): Number of teams per page
    - **Outputs:** JSON response containing a list of article nodes with basic information.
2. **GET** `admin/articleNode/{articleId}`
    - **Inputs:**
        - `articleId`: MongoDB unique identifier of the article node.
    - **Outputs:**JSON representation of the requested article node
3. **GET** `article/{_id}/check_like`
    - **Inputs:**
        - `_id`: MongoDB unique identifier of the article node.
    - **Outputs:** Boolean value to visualize if it is liked or not
4. **GET** `article/{_id}/count_like`
    - **Inputs:**
        - `_id`: MongoDB unique identifier of the article node.
    - **Outputs:** Number of likes counted
5. **POST** `admin/map/articles`
    - **Inputs:**
    - **Outputs:** String with number of articles correctly mapped from MongoDB to Neo4j
6. **POST** `admin/map/wrote_relationships`
    - **Inputs:**
    - **Outputs:** String with number of relationships correctly created between users and articles

4.5.8 API Endpoints: Coaches Node Controller

1. **GET** `search/filter/coach/list/byGender/{gender}`
   - **Inputs:**
     - `gender`: gender attribute
     - `page`, `size`: Pagination controls.
   - **Outputs:** JSON response containing paginated coach node summaries gruped by gender.
2. **GET** `admin/coachNode/{_id}`
   - **Inputs:**
     - `_id`: Unique identifier of the coach node in Neo4j.
   - **Outputs:** JSON object with complete coach node information
3. **GET** `/coach/{_id}/team`
   - **Inputs:**
     - `_id`: Unique MongoDB identifier of the coach.
   - **Outputs:** JSON array listing all teams managed by the coach currently (last year)
4. **GET** `coach/{_id}/managing_history`
   - **Inputs:**
     - `_id`: Unique MongoDB identifier of the coach.
   - **Outputs:** JSON array listing all teams managed by the coach historically
5. **GET** `coach/{_id}/team/{fifaV}`
   - **Inputs:**
     - `_id`: Unique MongoDB identifier of the coach.
     - `fifaV`: Fifa version attribute
   - **Outputs:** JSON object cotaining details about the team managed by the coach in that year
6. **GET** `coach/{_id}/check_like`
   - **Inputs:**
     - `_id`: Unique identifier of the coach node in Neo4j.
   - **Outputs:** Boolean value to visualize if it is liked or not
7. **GET** `coach/{_id}/count_like`
   - **Inputs:**
     - `_id`: Unique identifier of the coach node in Neo4j.

- **Outputs:** Number of likes counted

8. **POST** `admin/map/coaches`
   - **Inputs:**
   - **Outputs:** String response containing the number of newly created coach node.
9. **POST** `/admin/map/manages_team_relationship/{gender}`
   - **Inputs:**
     - `gender`: gender attribute
   - **Outputs:** String response containing the number of newly created relationship between coach nodes and team nodes grouped by gender.

4.5.9 API Endpoints: Players Node Controller

1. **GET** `/admin/playerNode/{_id}`
   - **Inputs:**
     - `_id`: The unique MongoDB node identifier for the player.
   - **Outputs:** JSON representation of the player node
2. **GET** `/search/filter/player/list/byGender/{gender}`
   - **Inputs:**
     - `gender`: The gender to filter by
     - `page`, `size`: Pagination controls.
   - **Outputs:** JSON paginated result of player nodes matching the gender filter.
3. **GET** `/player/{_id}/team`
   - **Inputs:**
     - `_id`: The player's node MongoId.
   - **Outputs:** JSON object representing the team node the player is recently playing (last year)
4. **GET** `/player/{_id}/team/{fifaV}`
   - **Inputs:**
     - `_id`: Player node ID.
     - `fifaV`: FIFA version
   - **Outputs:** JSON of the team the player is playing for that year.
5. **GET** `player/{_id}/check_like`

- **Inputs:**
  - `_id`: The unique MongoDB node identifier for the player.
- **Outputs:** Boolean value to visualize if it is liked or not

6. **GET** `player/{_id}/count_like`

   - **Inputs:**
     - `_id`: The unique MongoDB node identifier for the player.
   - **Outputs:** Number of likes counted

7. **POST** `/admin/map/plays_relationships/{gender}`

   - **Inputs:**
     - `gender`: Gender to scope the mapping
   - **Outputs:** Success message with number of relationship mapped into Neo4j.

8. **POST** `/admin/map/players`

   - **Inputs:**
   - **Outputs:** Response String with number of nodes mapped into Neo4j

4.5.10 API Endpoints: Teams Node Controller

1. **GET** `search/filter/team/list/byGender/{gender}`

   - **Inputs:**
     - `gender`: gender attribute to group by
     - `page`, `size`: Standard pagination controls.
   - **Outputs:** JSON array with teams grouped by gender

2. **GET** `admin/teamNode/{_id}`

   - **Inputs:**
     - `_id`: MongoDB node ID for the team.
   - **Outputs:** JSON with all available team details

3. **GET** `team/{_id}/current_formation`

   - **Inputs:**
     - `_id`: MongoDB team node ID.
   - **Outputs:** JSON object containing the list of players playing in that team in the last year.

4. **GET** `team/{_id}/formation/{fifaV}`

- **Inputs:**
  - `_id`: MongoDB team node ID.
  - `fifaV` : Fifa version attribute
- **Outputs:** JSON object containing the list of players playing in that team in the specific year.

5. **GET** `team/{_id}/coach`

   - **Inputs:**
     - `_id`: MongoDB team node ID.
   - **Outputs:** JSON object containing the informations about the Coach node managing the team in the last year

6. **GET** `team/{_id}/coach/{fifaV}`

   - **Inputs:**
     - `_id`: MongoDB team node ID.
   - **Outputs:** JSON object containing the informations about the Coach node managing the team in the specific year

7. **GET** `team/{_id}/check_like`

   - **Inputs:**
     - `_id`: MongoDB node ID for the team.
   - **Outputs:** Boolean value to visualize if it is liked or not

8. **GET** `team/{_id}/count_like`

   - **Inputs:**
     - `_id`: MongoDB node ID for the team.
   - **Outputs:** Number of likes counted

9. **POST** `admin/map/teams`

   - **Inputs:**
   - **Outputs:** String response with the number of TeamsNode mapped into Neo4j

4.5.11 API Endpoints: Users Node Controller

1. **GET** `/admin/user/node/list`
   - **Inputs:**
     - `page`: page number for pagination
     - `size`: number of items per page
   - **Outputs:** JSON Page of details about UsersNode
2. **GET** `/user/{userName}/articles`
   - **Inputs:**
     - `userName`: the username of the user
     - `page`: page number for pagination
     - `size`: number of items per page
   - **Outputs:** JSON Page of details about ArticlesNode coressponding to the user
3. **GET** `user/{_id}`
   - **Inputs:**
     - `_id`: MongoDB identifier
   - **Outputs:** JSON Object with details about user, like email and nationality
4. **GET** `/user/team/male/getStats`
   - **Inputs:**
   - **Outputs:** List of FifaStatsPlayer of players the user has in male team
5. **GET** `/user/team/female/getStats`
   - **Inputs:**
   - **Outputs:** List of FifaStatsPlayer of players the user has in female team
6. **GET** `/user/team/male`
   - **Inputs:**
   - **Outputs:** List of details about PlayersNode in male team
7. **GET** `/user/team/female`
   - **Inputs:**
   - **Outputs:** List of details about PlayersNode in female team
8. **GET** `/user/{userName}/followings`
   - **Inputs:**
     - `userName`: username of the user

- **Outputs:** List of UsersNode the user is following

9. **GET** `/user/{userName}/followers`
   - **Inputs:**
     - `userName`: username of the user
   - **Outputs:** List of UsersNode the user is followed by
10. **GET** `user/{_id}/check_follow`
    - **Inputs:**
      - `_id`: MongoId of the user
    - **Outputs:** Boolean value to visualize if it followed or not
11. **GET** `/user/{userName}/followersAndFollowingsCounts`
    - **Inputs:**
      - `target`: username to get counts for
    - **Outputs:** JSON object with number of followers and followings
12. **POST** `/admin/map/users`
    - **Inputs:**
    - **Outputs:** Status message string with number of nodes mapped into Neo4j
13. **POST** `/article/{articleId}/like`
    - **Inputs:**
      - `articleId`: ID of the article to like
    - **Outputs:** Async HTTP status
14. **POST** `/user/team/male/addPlayer/{_id}/{fifaValue}`
    - **Inputs:**
      - `_id`: player MongoDB ID
      - `fifaValue`: season/year to use
    - **Outputs:** Async HTTP status
15. **POST** `/user/team/female/addPlayer/{_id}/{fifaValue}`
    - **Inputs:**
      - `_id`: player MongoDB ID
      - `fifaValue`: season/year to use
    - **Outputs:** Async HTTP status
16. **POST** `/team/{_id}/like`
    - **Inputs:**

    - `_id`: team MongoDB ID
    - Authenticated user: taken from session/token
  - **Outputs:** None (void)

17. **POST** `/player/{_id}/like`
  - **Inputs:**
    - `_id`: player MongoDB ID
  - **Outputs:** Async HTTP status

18. **POST** `/coach/{_id}/like`
  - **Inputs:**
    - `_id`: coach MongoDB ID
  - **Outputs:** Async HTTP status

19. **PUT** `/admin/populate/likes_to_players`
  - **Inputs:**
  - **Outputs:** Status message string

20. **PUT** `/admin/populate/likes_to_teams`
  - **Inputs:**
  - **Outputs:** Status message string

21. **PUT** `/admin/populate/likes_to_coaches`
  - **Inputs:**
  - **Outputs:** Status message string

22. **PUT** `/admin/populate/follows`
  - **Inputs:** None
  - **Outputs:** Status message string

23. **PUT** `/user/{userName}/follow`
  - **Inputs:**
    - `target`: username to follow
  - **Outputs:** Async HTTP stattus

24. **DELETE** `/user/team/male/removePlayer/{_id}`
  - **Inputs:**
    - `_id`: player MongoDB ID
  - **Outputs:** Async HTTP stattus

25. **DELETE** `/user/team/female/removePlayer/{_id}`

- **Inputs:**
  - `_id`: player MongoDB ID
- **Outputs:** Async HTTP stattus

26. **DELETE** `/user/{target}/unfollow`
    - **Inputs:**
      - `target`: username to unfollow
    - **Outputs:** Async HTTP stattus
27. **DELETE** `/article/{articleId}/unlike`
    - **Inputs:**
      - `articleId`: ID of the article to unlike
    - **Outputs:** Async HTTP stattus
28. **DELETE** `/team/{_id}/unlike`
    - **Inputs:**
      - `_id`: team MongoDB ID
    - **Outputs:** Async HTTP stattus
29. **DELETE** `/player/{_id}/unlike`
    - **Inputs:**
      - `_id`: player MongoDB ID
    - **Outputs:** Async HTTP stattus
30. **DELETE** `/coach/{_id}/unlike`
    - **Inputs:**
      - `_id`: coach MongoDB ID
    - **Outputs:** Async HTTP stattus

4.5.12 API Endpoints: NodeController

1. **GET** `admin/analytics/team/topByFame/{fifaVersion}`
    - **Inputs:**
      - `fifaVersion`: Integer indicating the FIFA version to evaluate
    - **Outputs:** JSON array of the most famous team(s) for the given version
    - **Example:**
      (a) **Input:** 24
      (b) **Output:**

```
          [
{
  "teamNode": {
    "mongoId": "682493307b8223979a59b9f8",
    "longName": "Derby County",
    "gender": "male",
    "fifaVersion": 24
  },
  "totalFame": 5
},
{
  "teamNode": {
    "mongoId": "682493307b8223979a59b8b1",
    "longName": "Rayo Vallecano",
    "gender": "male",
    "fifaVersion": 24
  },
  "totalFame": 3
},
{
  "teamNode": {
    "mongoId": "682493307b8223979a59ba07",
    "longName": "Rotherham United",
    "gender": "male",
    "fifaVersion": 24
  },
  "totalFame": 3
},
{
  "teamNode": {
    "mongoId": "682493307b8223979a59b9af",
    "longName": "Al Fateh",
    "gender": "male",
    "fifaVersion": 24
  },
  "totalFame": 3
},
{
  "teamNode": {
    "mongoId": "682493307b8223979a59b8d0",
    "longName": "Monza",
    "gender": "male",
    "fifaVersion": 24
  },
  "totalFame": 3
},
{
  "teamNode": {
```

```
        "mongoId": "682493307b8223979a59ba36",
        "longName": "Saarbrücken",
        "gender": "male",
        "fifaVersion": 24
      },
      "totalFame": 3
    },
    {
      "teamNode": {
        "mongoId": "682493307b8223979a59b8dc",
        "longName": "Las Palmas",
        "gender": "male",
        "fifaVersion": 24
      },
      "totalFame": 3
    },
    {
      "teamNode": {
        "mongoId": "682493307b8223979a59b8b9",
        "longName": "Getafe",
        "gender": "male",
        "fifaVersion": 24
      },
      "totalFame": 3
    },
    {
      "teamNode": {
        "mongoId": "682493307b8223979a59b94d",
        "longName": "Le Havre",
        "gender": "male",
        "fifaVersion": 24
      },
      "totalFame": 3
    },
    {
      "teamNode": {
        "mongoId": "682493307b8223979a59b9f0",
        "longName": "Austin",
        "gender": "male",
        "fifaVersion": 24
      },
      "totalFame": 2
    }
  ]
```


2. **GET** `admin/analytics/player/mostEngaged`

   - **Inputs:**

- **Outputs:** JSON array with the most engaged player
- **Example:**
  (a) **Input:** Nothing
  (b) **Output:**


```
                [
    {
      "player": {
        "mongoId": "68249ab17b8223979a59f084",
        "longName": "Michael Jhon Ander Rangel Valencia",
        "gender": "male",
        "fifaV": null,
        "nationality_name": "Colombia"
      },
      "totalEngagement": 9
    }
  ]
```


3. **GET** `admin/analytics/user/interestDiversity`

- **Inputs:**
- **Outputs:** JSON array measuring user interest diversity
- **Example:**
  (a) **Input:** Nothing
  (b) **Output:**


```
                [
    {
      "userName": "Jake Lewis",
      "avgInterestCount": 66.33333333333333
    },
    {
      "userName": "Mia Eriksson",
      "avgInterestCount": 15
    },
    {
      "userName": "Nick Hartland",
      "avgInterestCount": 14.666666666666666
    },
    {
      "userName": "Francisco RicoAdria Soldevilla",
      "avgInterestCount": 14.666666666666666
    },
    {
      "userName": "Charles Watts",
      "avgInterestCount": 14.333333333333334
    },
    {
```

```
    "userName": "Adam Smith",
    "avgInterestCount": 14.333333333333334
  },
  {
    "userName": "Gareth Boyes",
    "avgInterestCount": 13.666666666666666
  },
  {
    "userName": "Goal",
    "avgInterestCount": 13.333333333333334
  },
  {
    "userName": "Matt O'Connor-Simpson",
    "avgInterestCount": 13
  },
  {
    "userName": "Adam Barnish",
    "avgInterestCount": 13
  }
]
```


4.5.13 API Endpoints: FootBallController

1. **GET** `admin/analytics/signupsSummary/{year}`

   - **Inputs:**
     - `year`: Integer representing the year to retrieve subscription summaries for
   - **Outputs:** JSON array with monthly subscription summary data
   - **Example:**
     (a) **Input:** 2021
     (b) **Output:**


```
[
  {
    "month": "January",
    "newSubscribers": 5
  },
  {
    "month": "February",
    "newSubscribers": 8
  },
  {
    "month": "March",
    "newSubscribers": 9
  },
  {
```

```
    "month": "April",
    "newSubscribers": 6
  },
  {
    "month": "May",
    "newSubscribers": 6
  },
  {
    "month": "June",
    "newSubscribers": 8
  },
  {
    "month": "July",
    "newSubscribers": 9
  },
  {
    "month": "August",
    "newSubscribers": 8
  },
  {
    "month": "September",
    "newSubscribers": 7
  },
  {
    "month": "October",
    "newSubscribers": 10
  },
  {
    "month": "November",
    "newSubscribers": 2
  },
  {
    "month": "December",
    "newSubscribers": 5
  }
]
```


2. **GET** `user/team/dreamTeam/{fifaV}`

   - **Inputs:**
     - `fifaV`: Integer specifying the FIFA version to get the dream team for
   - **Outputs:** JSON array of players forming the dream team for the specified FIFA version
   - **Example:**
     (a) **Input:** 21
     (b) **Output:**

```
          [
{
  "playerName": "Stine Pettersen Reinås",
  "overall": 92,
  "position": "CB"
},
{
  "playerName": "Delphine Cascarino",
  "overall": 93,
  "position": "RM, RW"
},
{
  "playerName": "Chloe Susan Arthur",
  "overall": 91,
  "position": "CM, CDM"
},
{
  "playerName": "Christiano Ronaldo",
  "overall": 90,
  "position": "CB, RB"
},
{
  "playerName": "Betsy Doon Hassett",
  "overall": 91,
  "position": "CAM, LM, RM"
},
{
  "playerName": "Marisa Isabel Van Der Meer",
  "overall": 87,
  "position": "LB, LWB"
},
{
  "playerName": "Aurora Cecilia Santiago Cisneros",
  "overall": 91,
  "position": "GK"
},
{
  "playerName": "Emily Gielnik",
  "overall": 93,
  "position": "RW, LW, ST"
},
{
  "playerName": "Nichelle Patrice Prince",
  "overall": 93,
  "position": "RM, ST"
},
{
  "playerName": "Stephanie Skilton",
```

```
    "overall": 83,
    "position": "RW, RWB, ST"
  },
  {
    "playerName": "Chloe Susan Arthur",
    "overall": 91,
    "position": "CM, CDM"
  }
]
```


3. **GET** `admin/team/{_id}/analytics/improvement/{year1}/{year2}`

   - **Inputs:**
     - `_id`: String team identifier (team ID)
     - `year1`, `year2`: Strings representing the two years to compare improvements
     - **Example:**
       (a) **Input:**
       _id : 6836e721ae93aaefe53bddb3,
       year1 : 2023,
       year2 : 2024
       (b) **Output:**


```
{
  "attImp": 3.0769231,
  "defImp": 0,
  "midImp": -1.5151515
}
```


   - **Outputs:** JSON object with percentage improvements in attack, defense, and midfield between the two years

4.5.14 Git-Hub

The above implementation can be found at the following link:

https://github.com/EnomisLP/Football/tree/main

Chapter 5

Future Improvements

The system will evolve to include key enhancements aimed at increasing intelligence, scalability, and user engagement. One of the primary additions will be a system-level feature for providing **automated suggestions**, such as recommending users or players to add to a team—based on behavioral patterns or content similarity.
User profiles will be extended to include additional attributes, enabling more contextualized features and future communication tools. As part of the roadmap, the platform will also introduce the ability for users to comment on articles, enriching the content with discussion and feedback. Other additional features that could be implemented are: providing data about the matches, championships. Also introducing verified accounts for teams, coaches and players, in order to have a platform that could be considered a reference point among in football world. Although the **front end** has not yet been implemented, it will be designed to accommodate these evolving capabilities.
Scalability will be addressed by implementing **sharding** on the database layer, allowing data to be distributed across multiple nodes and ensuring sustained performance under growing traffic and storage demands. Alongside these feature enhancements, the system will incorporate stronger security constraints to ensure data integrity, privacy, and resilience against common attack vectors.

Bibliography

[1] Khachin Borjigin, https://sofifa.com

[2] Khachin Borjigin, "SoFIFA Case Study with Sportmonks Football API", https://www.sportmonks.com/blogs/casestudy/sofifa/

[3] "The European Club Talent and Competition Landscape 2024"

[4] Warwas, Izabela and Dzimińska, Małgorzata and Krzewińska, Aneta, 2021, "The Frequency of Using Websites and Social Media by Various Age Groups to Form Opinions about Scientific Topics: Findings from the European Context", https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Young_people_-_digital_world#:~:text=Social%20networks%20were%20used%20in,Finland%20to%208%25%20in%20Romania.

[5] Eurostat, February 2025, "Population structure and ageing", https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Population_structure_and_ageing#:~:text=:%20Eurostat%20(demo_pjangroup)-,The%20share%20of%20the%20population%20aged%2065%20years%20and%20over,top%27%20of%20the%20population%20pyramid.

[6] YouGov in collaboration with Strive Sponsorship, November 2020, "The Many Faces of Sports Fans Across Europe Report", https://strivesponsorship.com/wp-content/uploads/2020/11/The-Many-Faces-of-Sports-Fans-Across-Europe-Report-2020.pdf

[7] "The State of Social", https://vidmob.com/resource/the-state-of-social

[8] "Understanding Neo4j's data on disk", https://neo4j.com/developer/kb/understanding-data-on-disk/

[9] www.semrush.com/analytics/traffic/overview

[10] https://www.k6agency.com/facebook-statistics/#part2

[11] https://www.socialinsider.io/social-media-benchmarks

[12] https://www.brandwatch.com/blog/twitter-stats-and-statistics/#:~:text=500%20million%20people%20visit%20the,the%20average%20number%20of%20followers

[13] https://explodingtopics.com/blog/x-user-stats#top